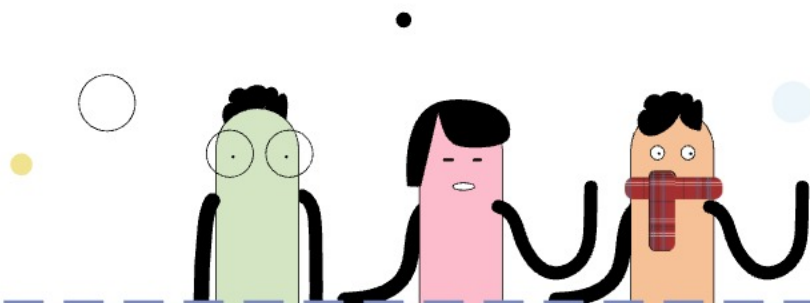




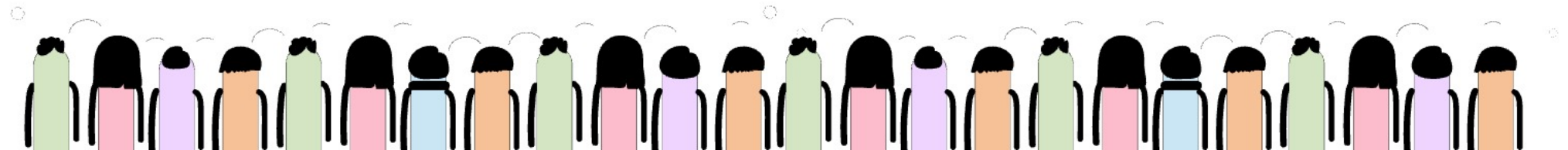
Advanced AI

Agents

★ Autonomous AI Infrastructure Platform 🌤️

An Autonomous AI Infrastructure Platform is:

A complete engineering platform where multiple AI agents can reason, retrieve data, communicate, make decisions, execute tasks, monitor themselves, and improve continuously with minimal human intervention.



★ Autonomous AI Infrastructure Platform 🌤️

Mapping Traditional OS → AI Runtime

Traditional OS Runtime

CPU Scheduler

Process Manager

Memory Allocator

Garbage Collector

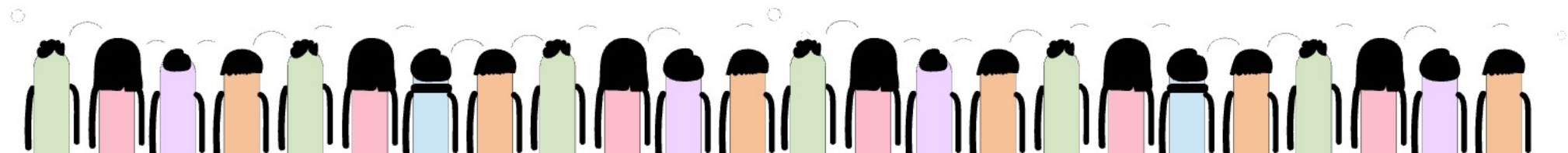
Autonomous AI Runtime

Agent Scheduler

Multi-Agent Coordinator

Context Window Allocator

Memory Compression / Context Eviction





Autonomous AI Infrastructure Platform

IPC

A2A Protocol

Thread Pool

Parallel Agent Execution

Async Runtime

Reasoning Execution Runtime

Kernel Events

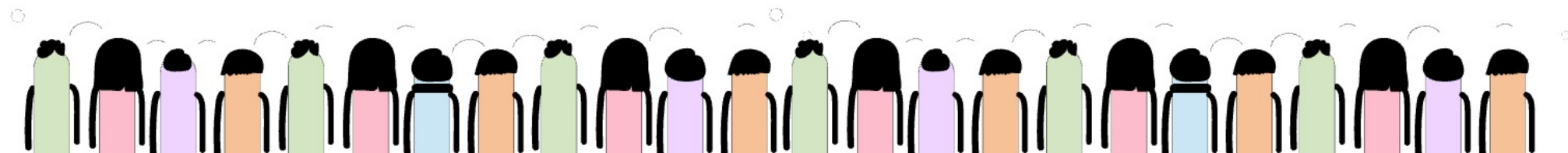
Agent Event System

Process Isolation

Agent Capability Isolation

Filesystem

Knowledge / Vector Memory Layer





Autonomous AI Infrastructure Platform

Layer	Innovation Weight
-------	-------------------

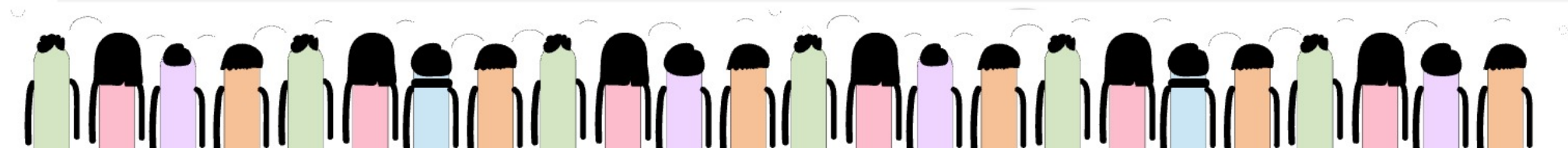
AI Runtime & Orchestration	30%
----------------------------	-----

Reasoning Compiler Systems	25%
----------------------------	-----

Memory & Retrieval Architecture	15%
---------------------------------	-----

AI Safety / Capability Systems	15%
--------------------------------	-----

Agent APIs / Tool Ecosystem	10%
-----------------------------	-----





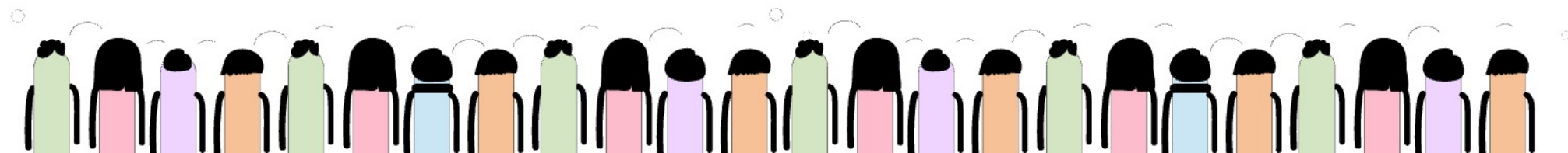
Autonomous AI Infrastructure Platform 🌤️

Memory & Retrieval Architecture 15%

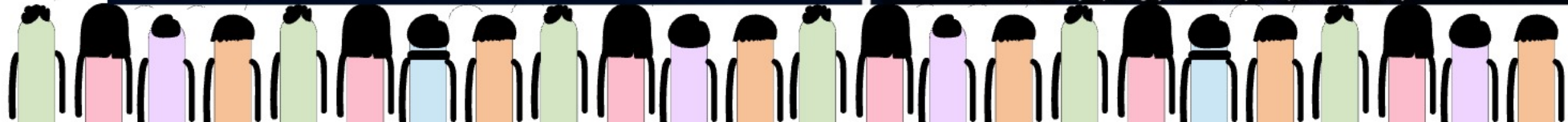
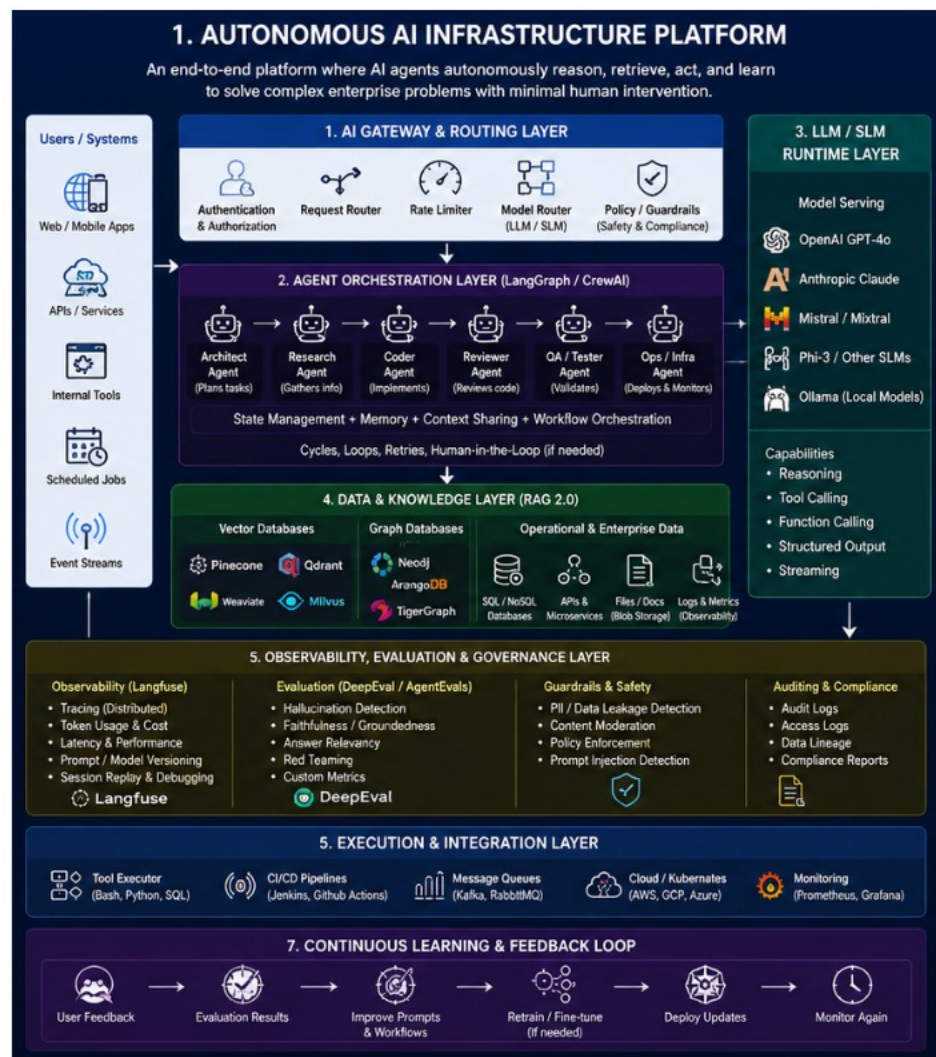
AI Safety / Capability Systems 15%

Agent APIs / Tool Ecosystem 10%

Prompt Engineering 5%



Autonomous AI Infrastructure Platform





Tools



Area

Tools

Agent orchestration

LangGraph, CrewAI

Prompt optimization

DSPy

Evaluation

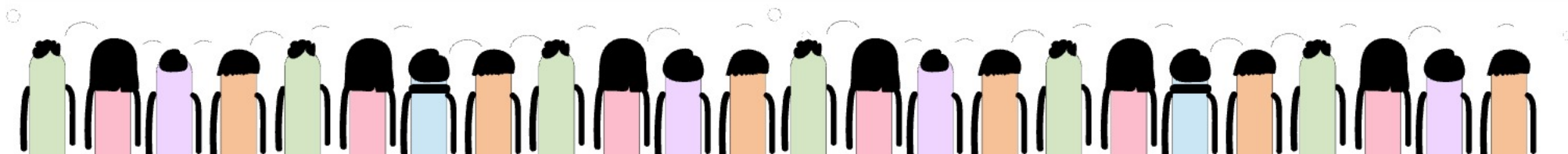
DeepEval, AgentEvals

Observability

Langfuse

Local LLMs

Ollama



Tools



RAG

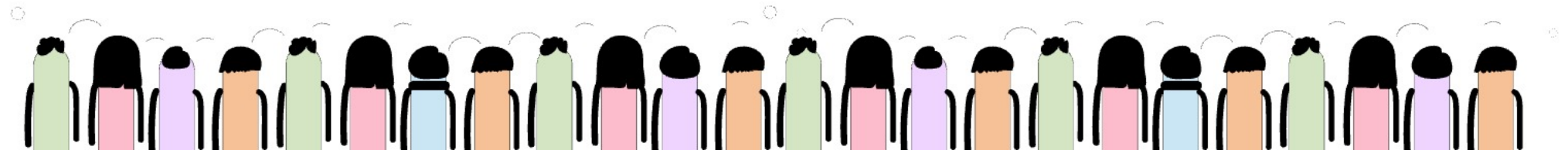
LlamaIndex, LangChain

Graph systems

Neo4j

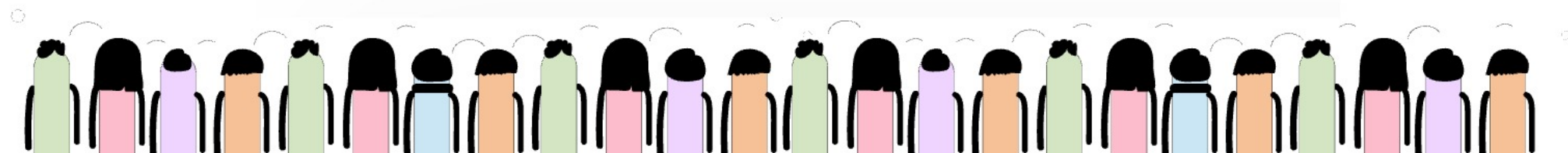
Vector DB

Qdrant, Weaviate, Pinecone



Infrastructure

Area	Needed
Docker	mandatory
Kubernetes	production AI deployment
GPU infra	inference serving
Redis	memory/cache
Kafka	event pipelines
FastAPI	serving agents
CI/CD	model deployment



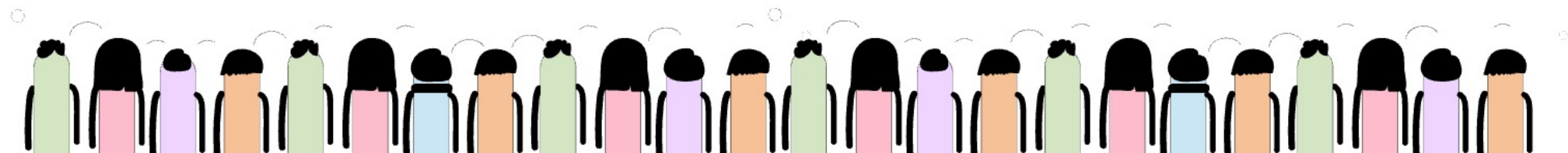


Gen AI Stack



Full Mapping of the Syllabus to the Real GenAI Innovation Stack

Innovation Layer	Weight	Syllabus Modules	What We Are Really Learning	Why It Matters in Production	
AI Runtime & Orchestration	30%	LangGraph, CrewAI, Agent Lightning, A2A	Autonomous execution systems	Multi-agent coordination at scale	

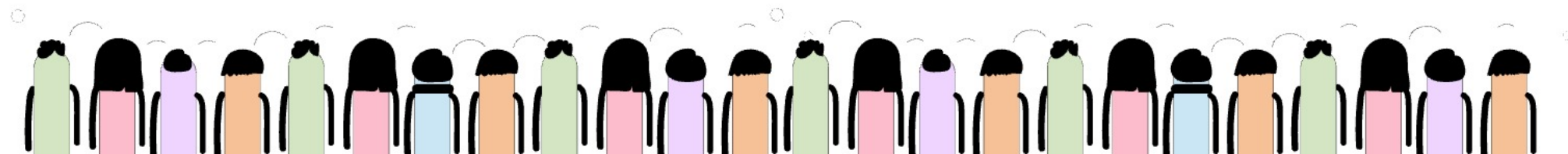




Gen AI Stack



Reasoning Compiler Systems	25%	DSPy, reasoning loops, cyclic graphs	Intent → execution graph compilation	Converts prompts into workflows
Memory & Retrieval Architecture	15%	GraphRAG, RAPTOR, HippoRAG	Long-term AI memory systems	Enterprise knowledge reasoning
AI Safety / Capability Systems	15%	DeepEval, Langfuse, hallucination detection	AI reliability engineering	Prevent unsafe or incorrect AI





Gen AI Stack



Agent APIs /
Tool
Ecosystem

10%

Ollama, LangChain,
Langfuse, A2A

AI
infrastructure
interoperability

Makes agents
operational

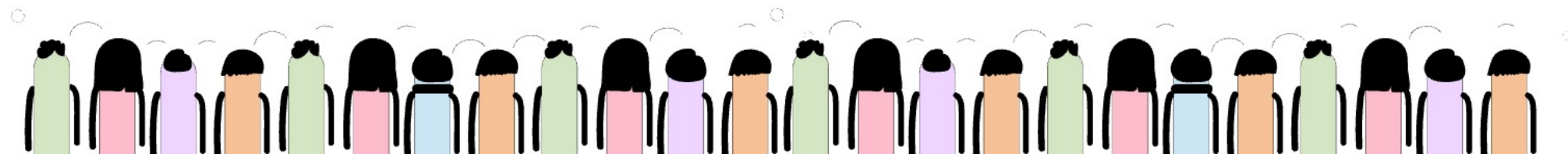
Prompt
Engineering

5%

DSPy, advanced
prompting

Surface-level
reasoning
optimization

Helps but not core
innovation

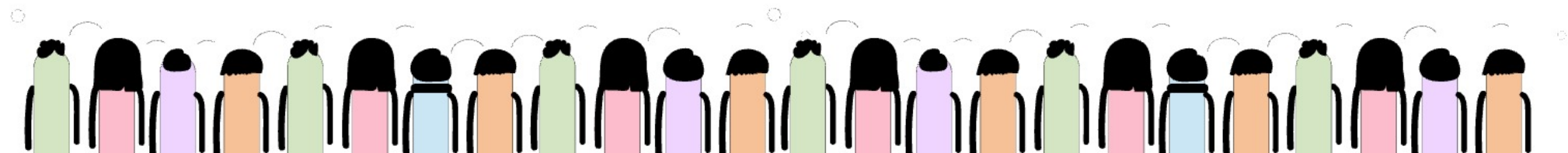




What is "Chunk + Embed"?

In most Retrieval-Augmented Generation systems:

1. Large documents are split into small pieces called **chunks**
2. Each chunk is converted into vectors using **embeddings**
3. User query is also embedded
4. Vector similarity search retrieves "closest chunks"
5. LLM generates answer from retrieved chunks

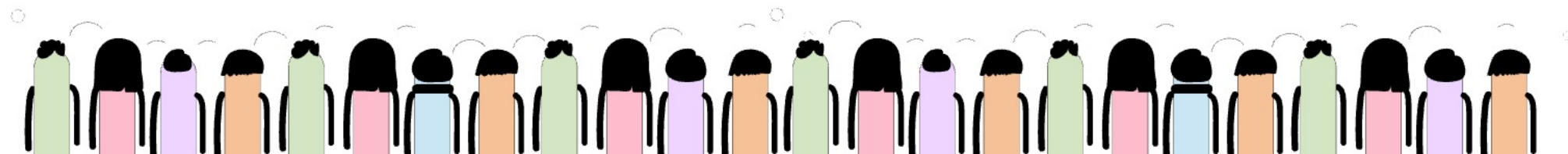




GOOD USE CASES

Use Case	Good?
Internal docs search	✓
HR policy chatbot	✓
PDF QA	✓
FAQ bots	✓
Small RAG systems	✓

↓





Because It Solves:

Problem

Solution

LLM doesn't know company data

Retrieve internal docs

Huge PDFs cannot fit context

Chunk into pieces

Fast semantic search needed

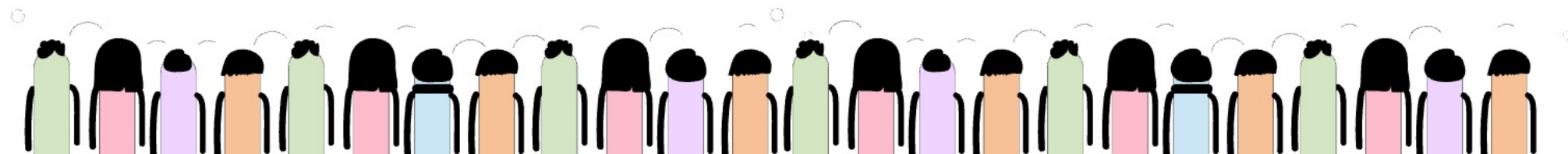
Use embeddings

Reduce hallucination

Ground answer from docs

Enterprise knowledge search

Vector retrieval

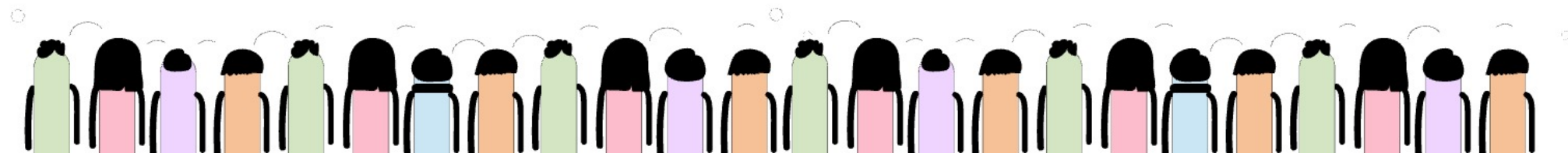




What is GraphRAG ?

GraphRAG =

Graph + RAG (Retrieval-Augmented Generation)



HOW GraphRAG WORKS



STEP 1 — Extract Entities

From documents:

"Kubernetes manages container workloads"

Extract:

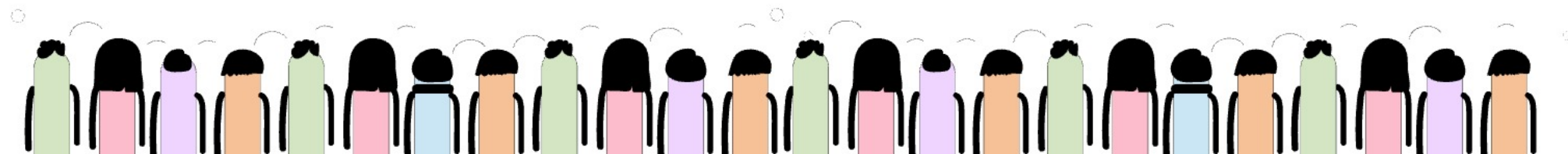
- Kubernetes
- containers
- workloads



STEP 2 — Build Relationships

Kubernetes

└─ manages ─→ containers





STEP 3 — Store as Graph

Usually in:

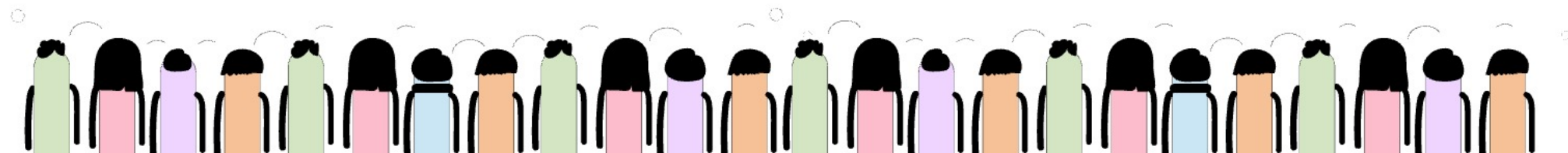
- Neo4j
- TigerGraph
- knowledge graph engines

STEP 4 — Query Understanding

User asks:

Which systems connected to overloaded API?

Graph traversal happens.





DOCUMENTS



+-----+
| Entity Extraction |
+-----+



+-----+
| Relationship Map |
+-----+



+-----+
| Knowledge Graph |
+-----+



User Query



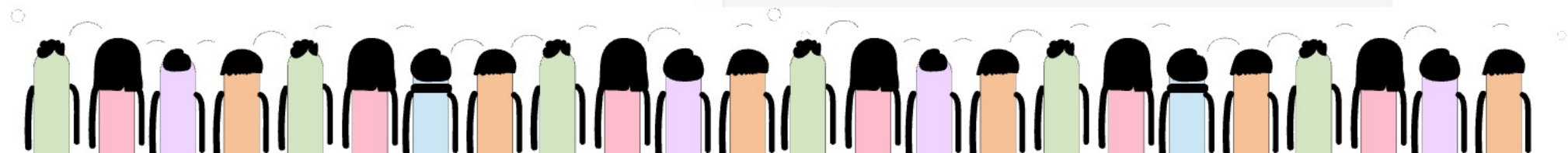
+-----+
| Graph Traversal |
+-----+



+-----+
| Connected Context |
+-----+



+-----+
| LLM Reasoning |
+-----+



WHERE GraphRAG IS USED?



1 Banking Fraud Systems

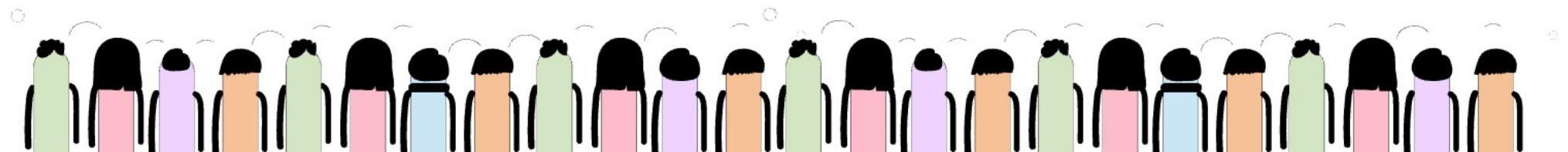
Example:

Customer
↓
Transactions
↓
Suspicious IP
↓
Shared Device
↓
Fraud Ring

2 Telecom Root Cause Analysis

Tower
↓
Network Link
↓
Router
↓
Service Failure

AI identifies outage chain.



3 Healthcare Systems



Patient



Symptoms



Medication



Side Effects

Helps medical reasoning.

GraphRAG vs Traditional RAG

Traditional RAG

Similar text search

Flat chunks

Semantic similarity

Weak dependency understanding

Good for documents

GraphRAG

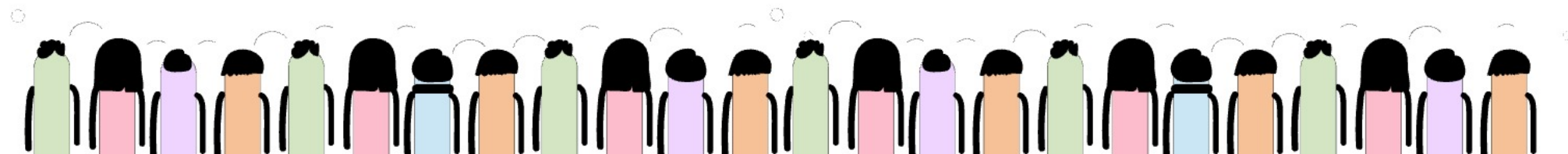
Relationship reasoning

Connected entities

Multi-hop intelligence

Strong dependency mapping

Good for enterprise systems





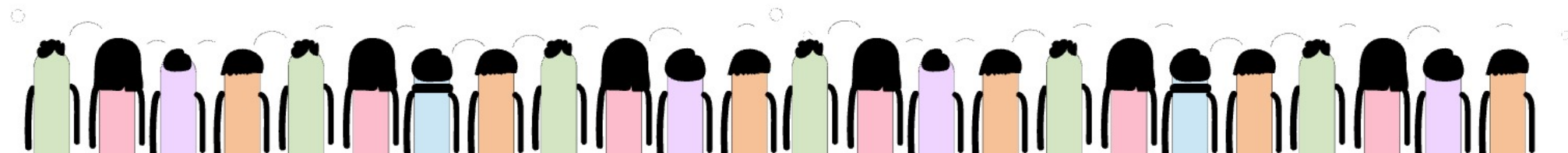
What is HippoRAG ?

HippoRAG =

Hierarchical Memory + Knowledge Graph + RAG

It is inspired by the:

Hippocampus





SIMPLE IDEA

Traditional Retrieval-Augmented Generation

Search similar chunks

GraphRAG:

Search relationships

HippoRAG:

Build associative long-term memory
like a human brain



WHY HippoRAG WAS CREATED?

Because traditional RAG has serious limitations:

Traditional RAG Problems

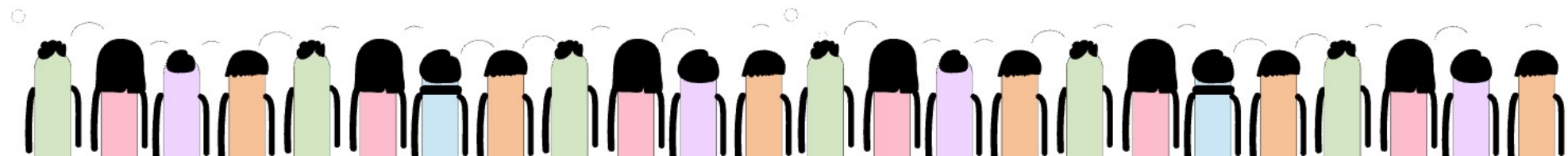
No long-term memory

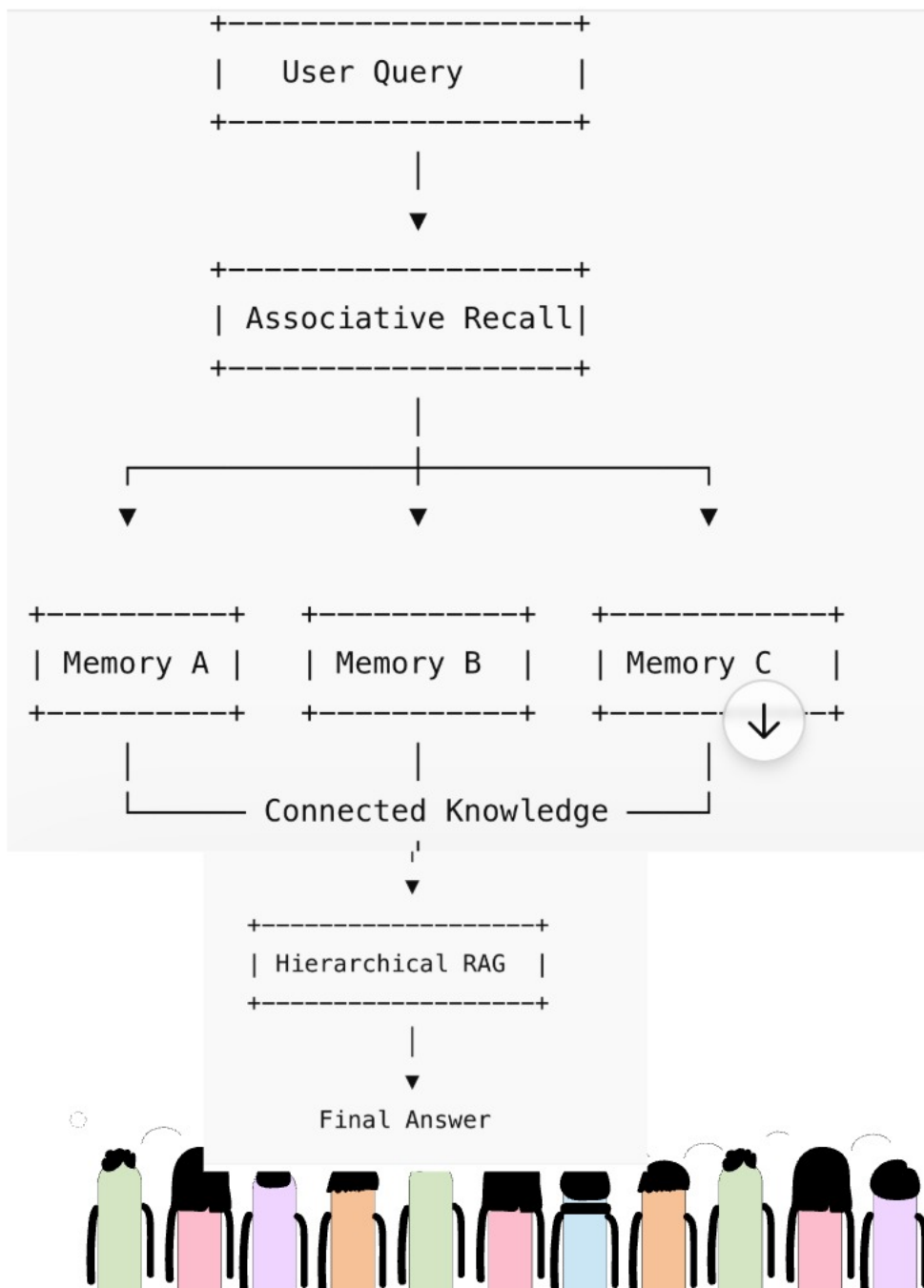
Weak multi-hop reasoning

Forgetful retrieval

Poor contextual association

No memory organization





HOW HippoRAG WORKS

STEP 1 — Extract Knowledge

Documents processed.

Example:

"Kubernetes manages scalable containers"

HOW HippoRAG WORKS

STEP 1 — Extract Knowledge

Documents processed.

Example:

"Kubernetes manages scalable containers"

STEP 3 — Create Associations

Build connected memory links:

Kubernetes
↓
Containers
↓
Microservices
↓
Scaling

STEP 2 — Build Memory Nodes

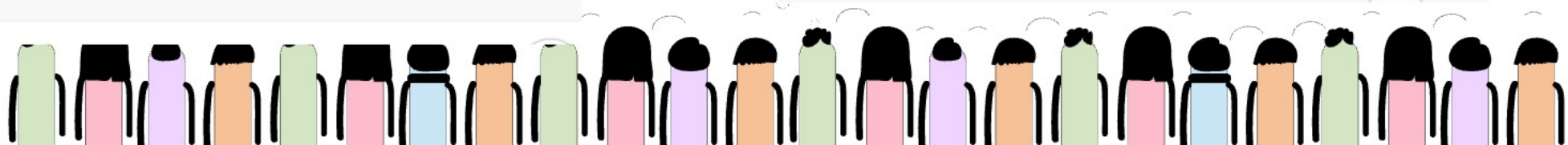
Create memory entities:

Kubernetes
Containers
Scalability

STEP 4 — Hierarchical Memory

Organize memory layers:

Concept
↓
Topic
↓
Domain
↓
System Knowledge



STEP 5 — Query Recall



User asks:

How do distributed systems scale?

HippoRAG recalls:

- Kubernetes
- load balancing
- Kafka
- autoscaling
- distributed computing

through associative traversal.

HippoRAG can associate:

Payment API



Database latency



Network congestion

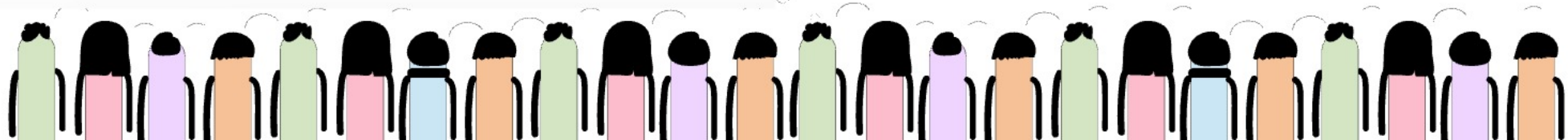


Kubernetes scaling issue

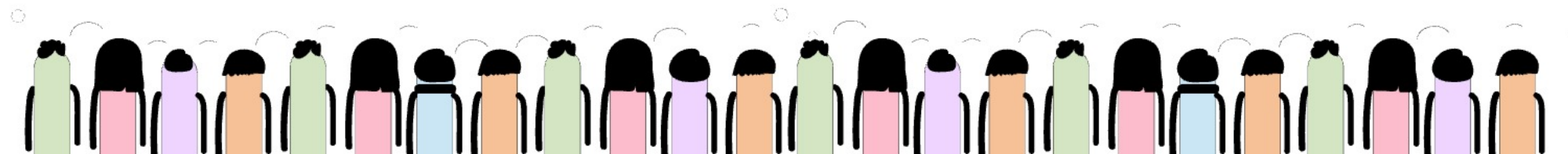
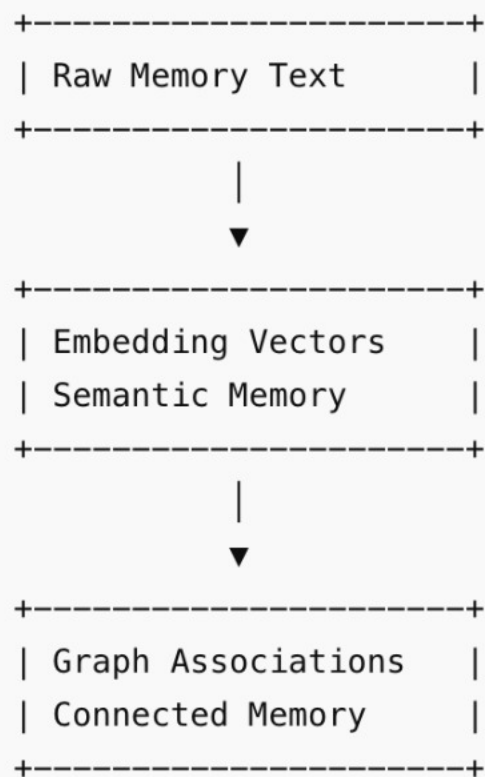


Recent deployment

This is MEMORY-DRIVEN reasoning.



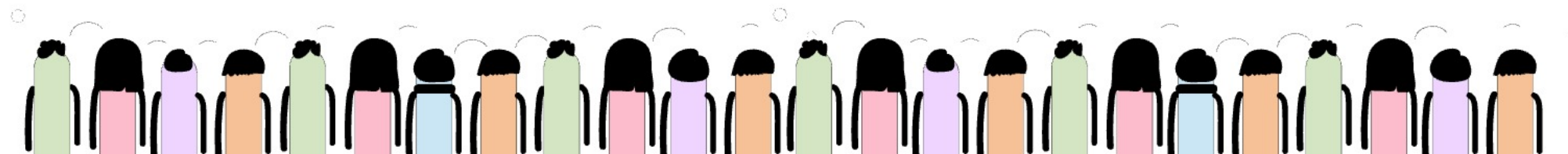
MEMORY STORAGE ARCHITECTURE





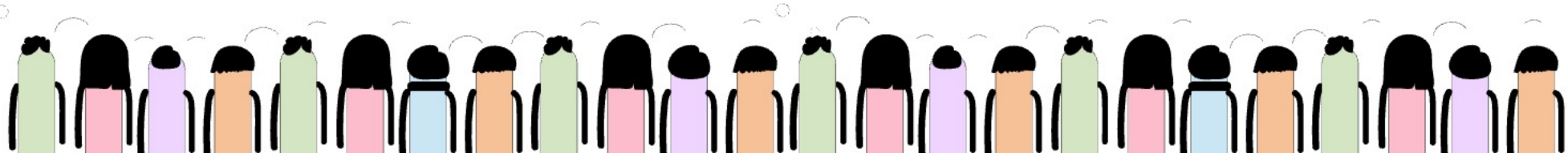
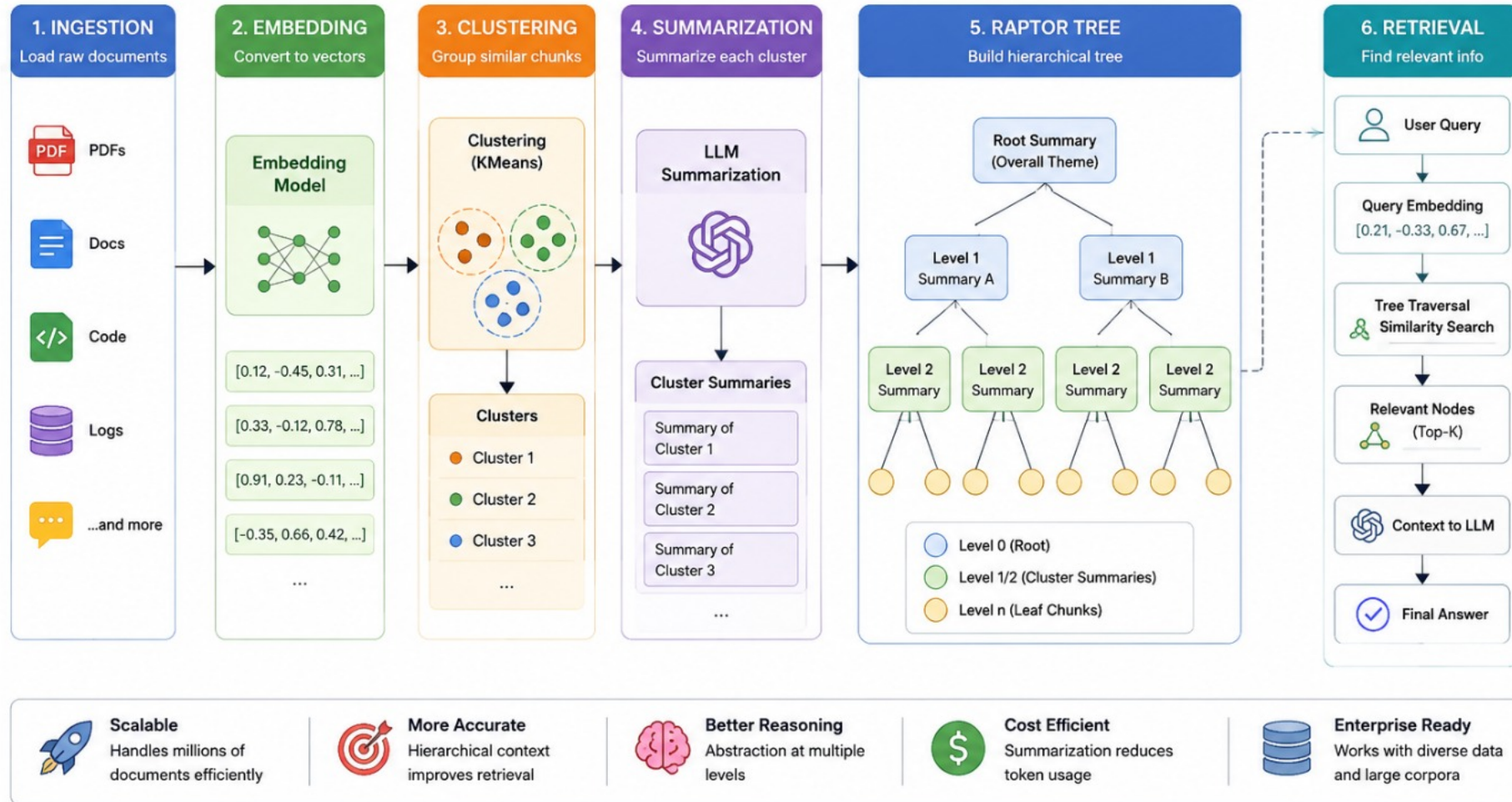
MEMORY TYPES IN YOUR CODE

Memory Layer	Stored Where
Raw memory	Python list
Semantic memory	embeddings
Associative memory	graph edges
Retrieved context	recalled list
Reasoning memory	LLM prompt



RAPTOR

Recursive Abstractive Processing for Tree-Organized Retrieval



STEP 1 — DOCUMENTS

```
</> Python  
  
documents = [...]
```

Enterprise knowledge.

Examples:

- Kubernetes
- Kafka
- scaling
- databases

STEP 2 — EMBEDDINGS

```
</> Python  
  
embedding_model.encode(texts)
```

Converts text into semantic vectors.

EXAMPLE

"Kubernetes manages containers"

becomes:

[0.442, -0.912, ...]

STEP 3 — CLUSTERING

```
</> Python  
  
KMeans(...)
```

Groups similar concepts together.

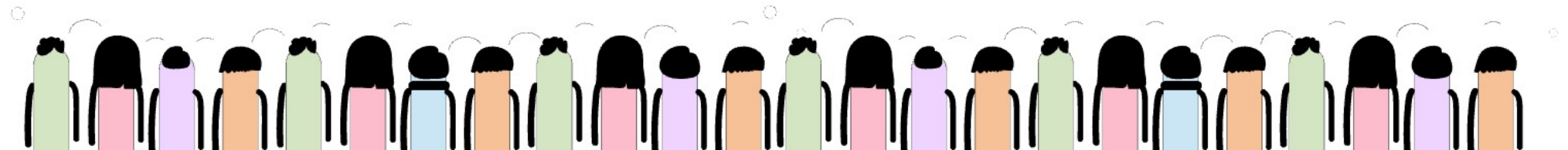
EXAMPLE CLUSTERS

Cluster 1:

- Kubernetes
- Autoscaling
- Load balancing

Cluster 2:

- Kafka
- Distributed systems
- Fault tolerance



STEP 4 — SUMMARIZATION

Python

```
summarize_cluster(...)
```

LLM compresses cluster into abstraction.

EXAMPLE

Input:

Kubernetes
Autoscaling
Load balancing

Output:

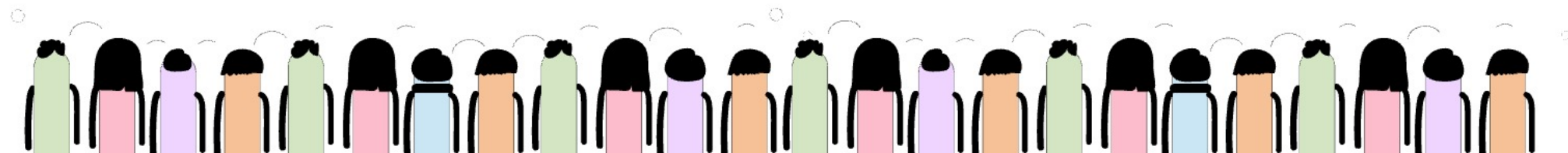
STEP 5 — BUILD TREE

Python

```
build_raptor_tree(...)
```

Creates:

Knowledge hierarchy



STEP 6 — QUERY

User asks:

"How do distributed systems scale?"

QUERY FLOW

Query



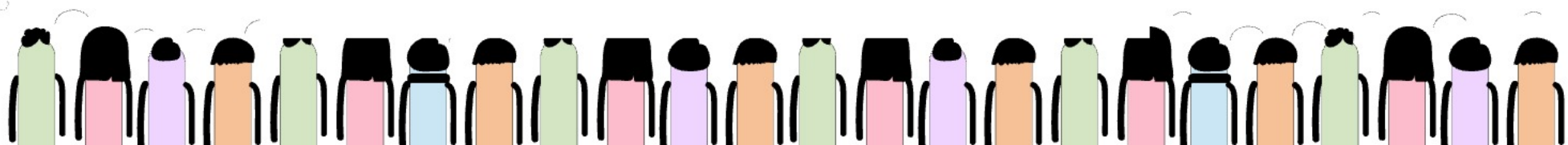
Embedding



Compare against summaries



Find best abstraction





REAL ENTERPRISE USE CASES

Industry

RAPTOR Usage

CloudOps

incident abstraction

Telecom

outage summarization

Banking

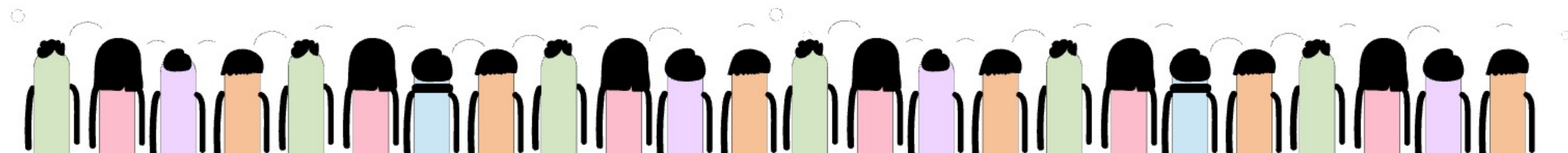
policy hierarchy

Research AI

paper abstraction

Enterprise Search

large document reasoning



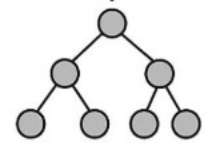
Documents

Chunking

Embedding

Clustering

Abstractive Summarization



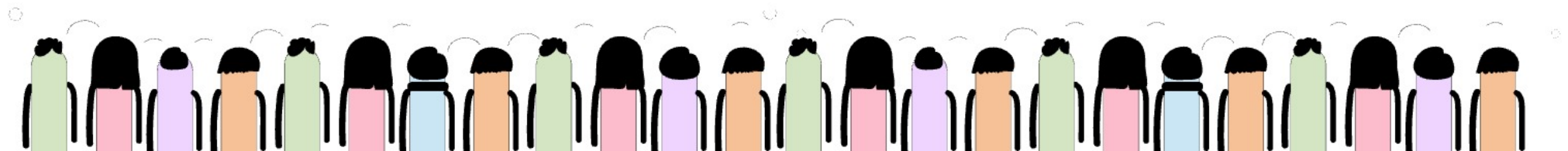
Hierarchical Summary
Tree (Multi-level)

Vector Store
(Embeddings for all leels)

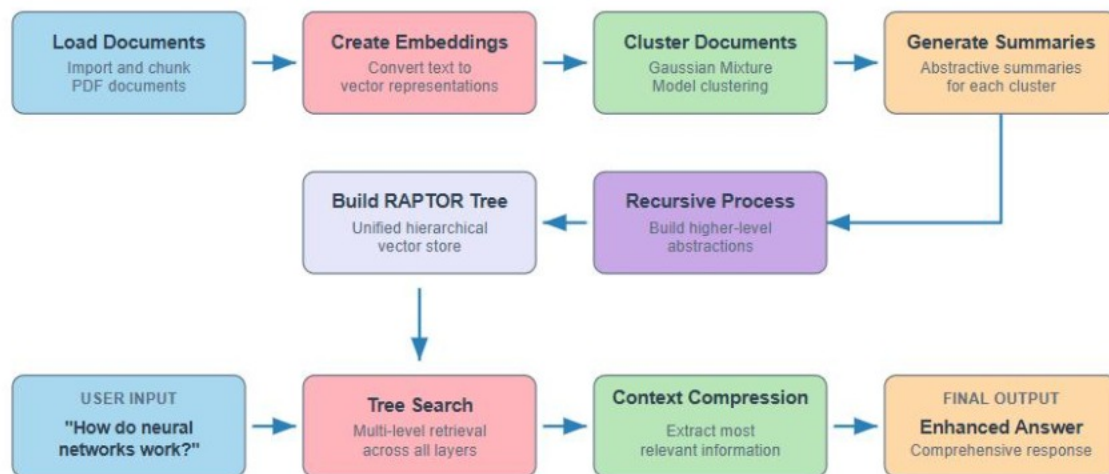
RAPTORRetriever
(recursive search)

Context Assembler

LLM
(via LangChain)



RAPTOR Workflow

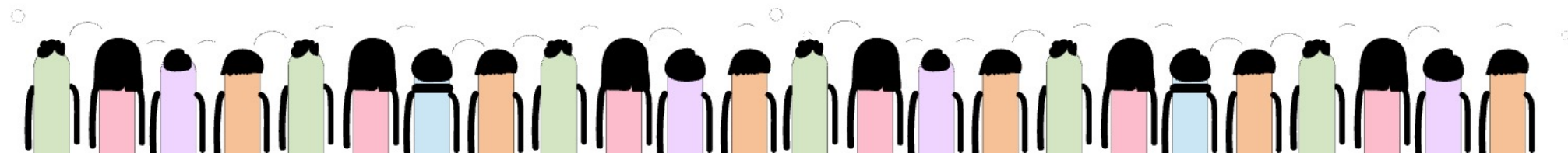


RAPTOR Components



Key Benefits:

- ✓ Seamless integration with hierarchical document organization
- ✓ Custom clustering algorithms and summarization strategies
- ✓ Recursive abstraction models for complex document structures
- ✓ Works with multi-level retrieval and tree search
- ✓ Easy to test, validate, and deploy in production
- ✓ Follows established patterns and best practices





START



Load documents



Generate embeddings



Cluster embeddings



Group similar documents



Send cluster to LLM



Generate summary



Create higher-level abstraction



Store summaries



User asks query



Embed query



Compare query vs summaries



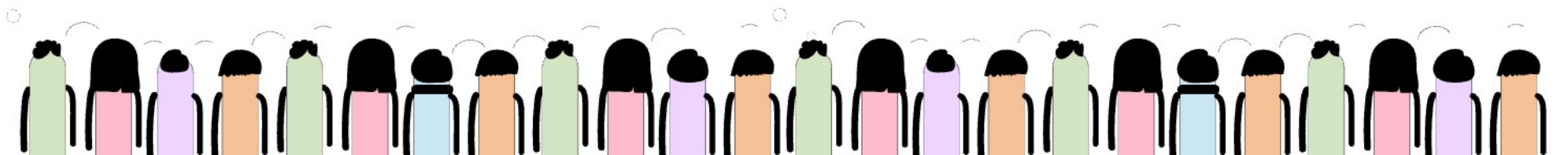
Find best matching summary



Return answer



END

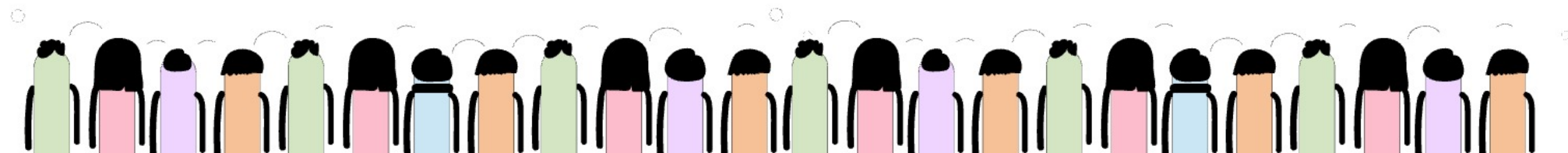




Why Small Language Models (SLMs)?

Examples:

- Microsoft Phi-3
- Mistral AI Mistral 7B

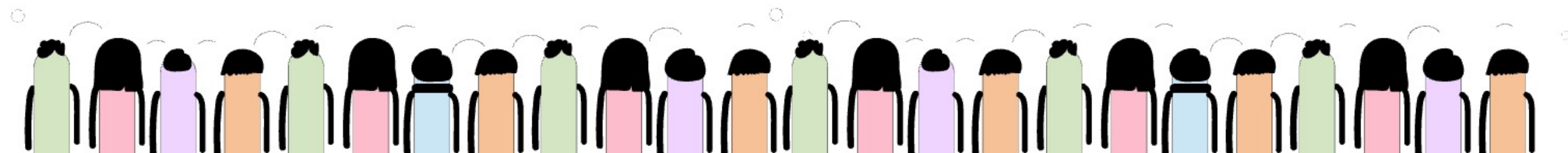




SIMPLE DEFINITION

SLM = Smaller AI model designed for:

- ✓ lower cost
- ✓ faster inference
- ✓ edge deployment
- ✓ lower memory usage
- ✓ specialized tasks



WHY SLMs BECAME IMPORTANT?

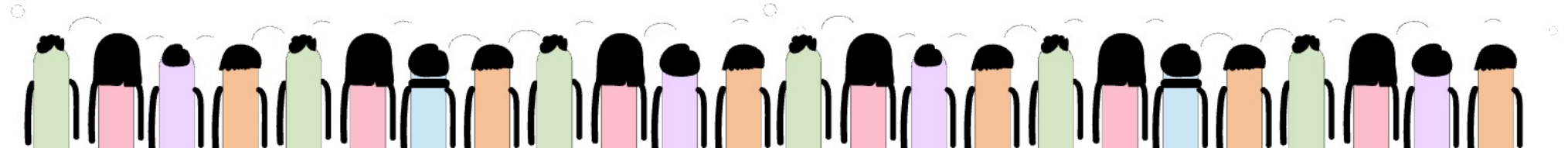


Large models like:

- GPT-4
- Claude
- Gemini

are powerful but:

- ✗ expensive
- ✗ huge GPU requirement
- ✗ high latency
- ✗ difficult for edge devices





LLM

Huge model

Expensive

Cloud-heavy

General-purpose

Higher latency

Massive GPU need

SLM

Compact model

Cost-efficient

Edge-friendly

Specialized

Faster response

Smaller hardware





WHY COMPANIES LOVE SLMs?

1 Lower Cost

Large LLMs:

High GPU bill

SLMs:

Much cheaper inference



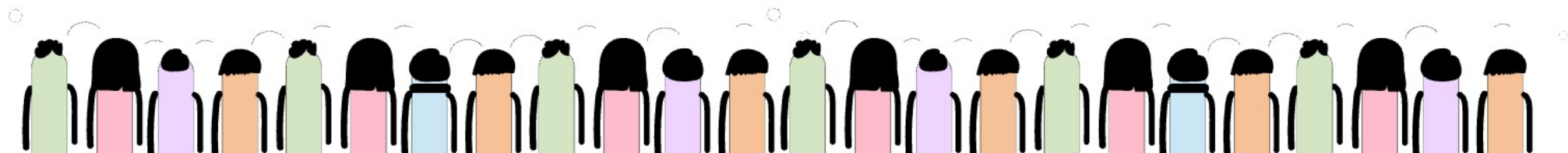
2 Faster Response

Smaller models:

- fewer parameters
- faster computation

Good for:

- realtime AI
- copilots
- assistants



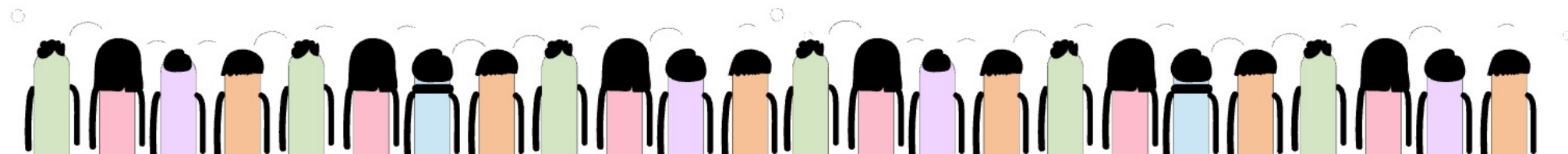


What is RAG?

Retrieval-Augmented Generation

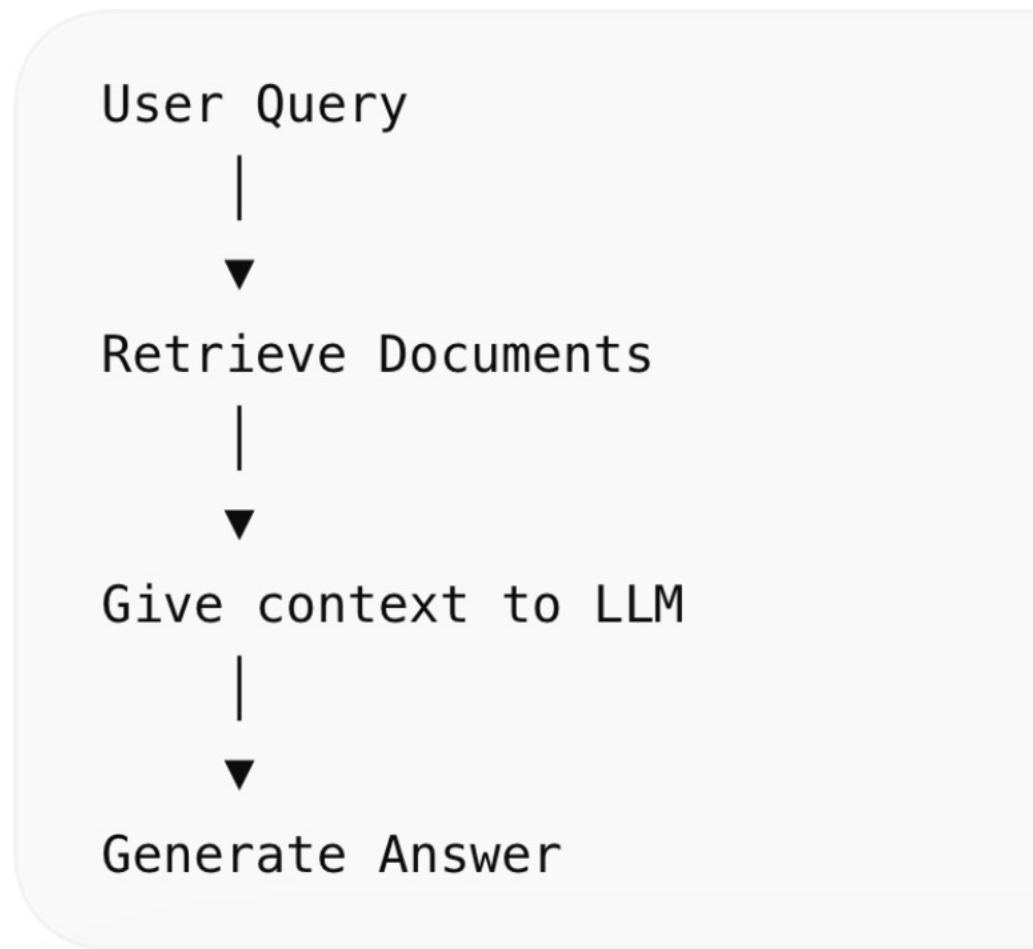
RAG means:

Retrieve external knowledge dynamically
instead of training model weights





SIMPLE RAG FLOW



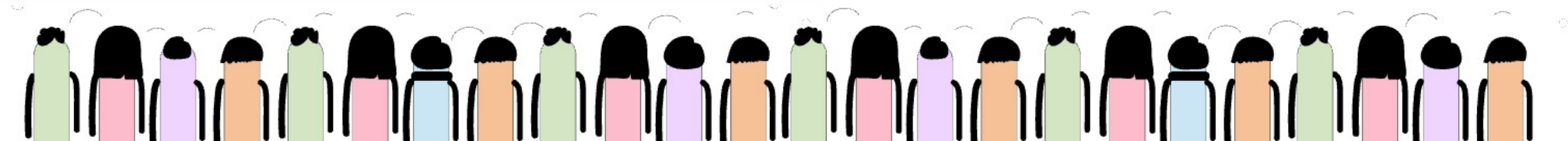
SIMPLE RULE

Use Fine-Tuning for:

HOW model behaves

Examples:

- response style
- formatting
- tool calling
- workflows





Fine-Tuning vs RAG

Fine-Tuning

Changes model weights

Permanent learning

Expensive training

Good for behavior

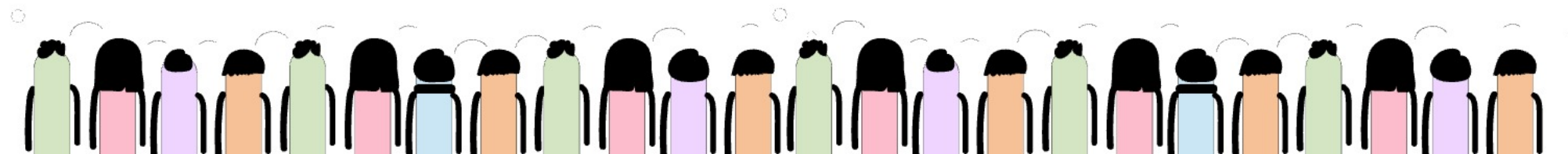
RAG

Uses external documents

Dynamic retrieval

Easier updates

Good for knowledge



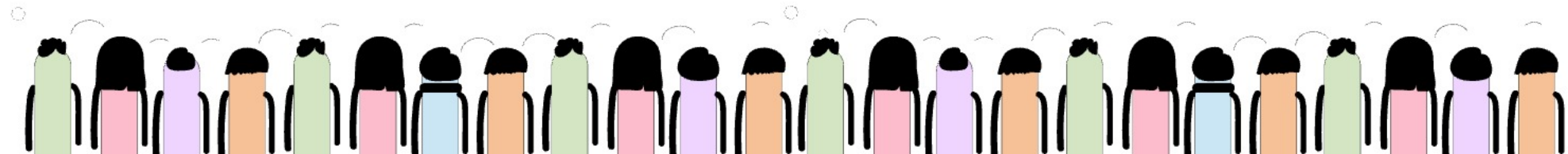
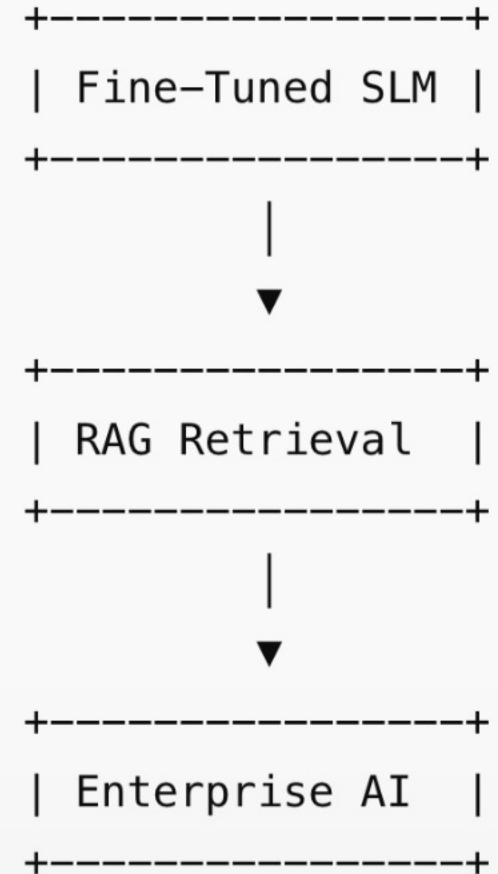
MODERN AI ARCHITECTURE

Use RAG for:

WHAT model knows

Examples:

- PDFs
- policies
- logs
- enterprise data
- live knowledge





User Query



SLM



Tool Selection



API Execution



Final Response

Industry

SLM Use

Banking

Fraud copilots

Telecom

Incident summaries

Healthcare

Medical note summarization

Manufacturing

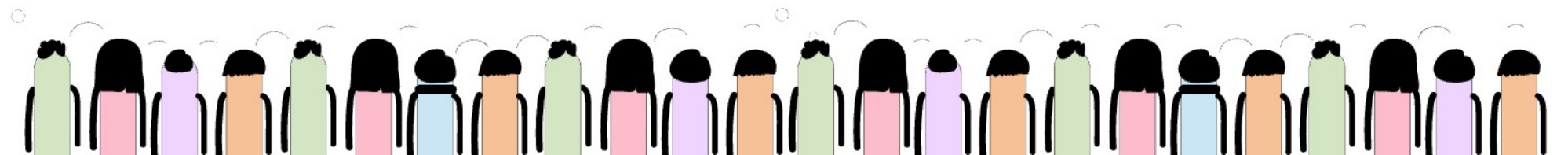
Edge maintenance AI

Automotive

In-vehicle assistants

Cybersecurity

Alert classification





Task

Model Type

Log summarization

Phi-3 / Mistral 7B

Root cause analysis

LLM + RAG

Alert classification

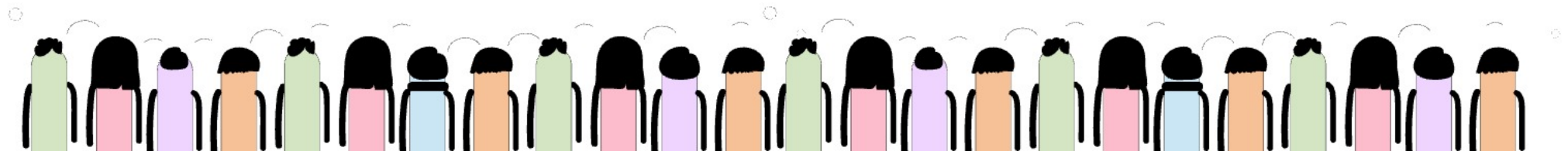
Fine-tuned SLM

Incident grouping

Embedding models

Anomaly detection

ML + time-series models



SIMPLE REAL EXAMPLE

Company logs:

```
[ERROR] API latency spike  
[WARN] CPU 95%  
[ERROR] Database timeout  
[INFO] Kubernetes pod restart
```

AI system should answer:

Root cause:
Database overload causing API retries
and Kubernetes pod instability.

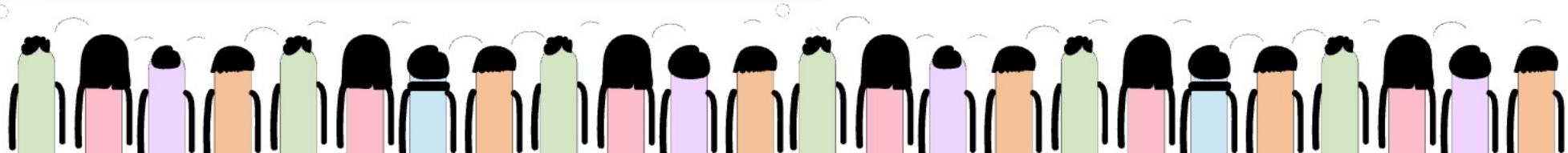
WHY NORMAL LLM IS NOT ENOUGH?

Raw LLM:

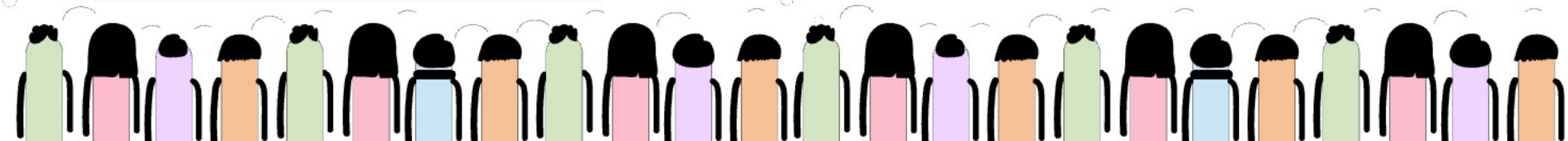
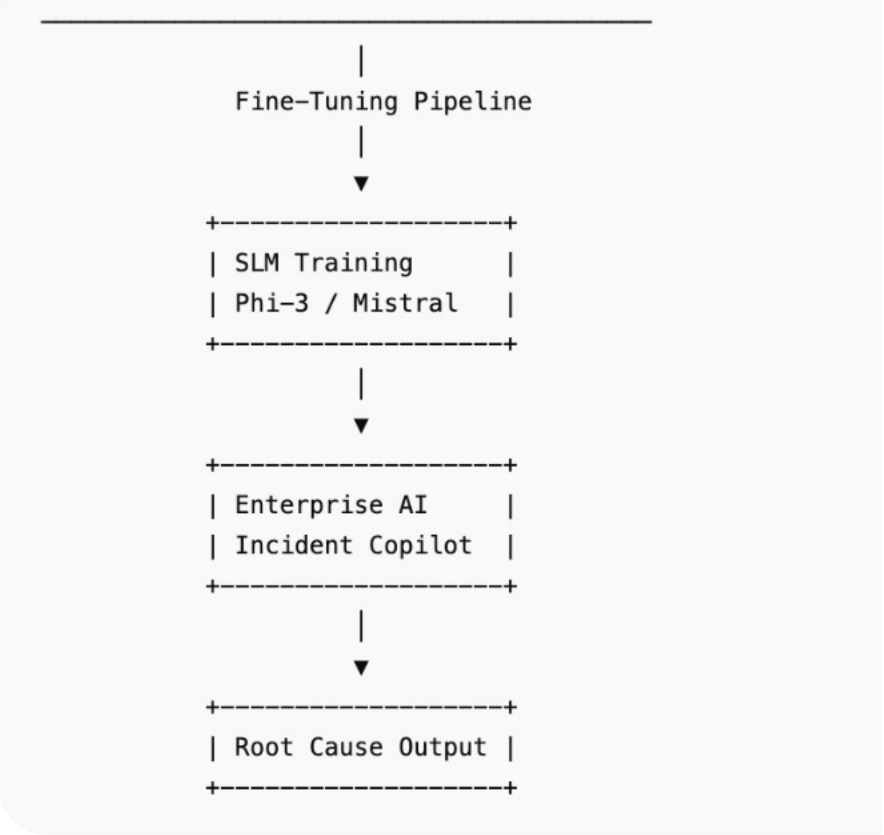
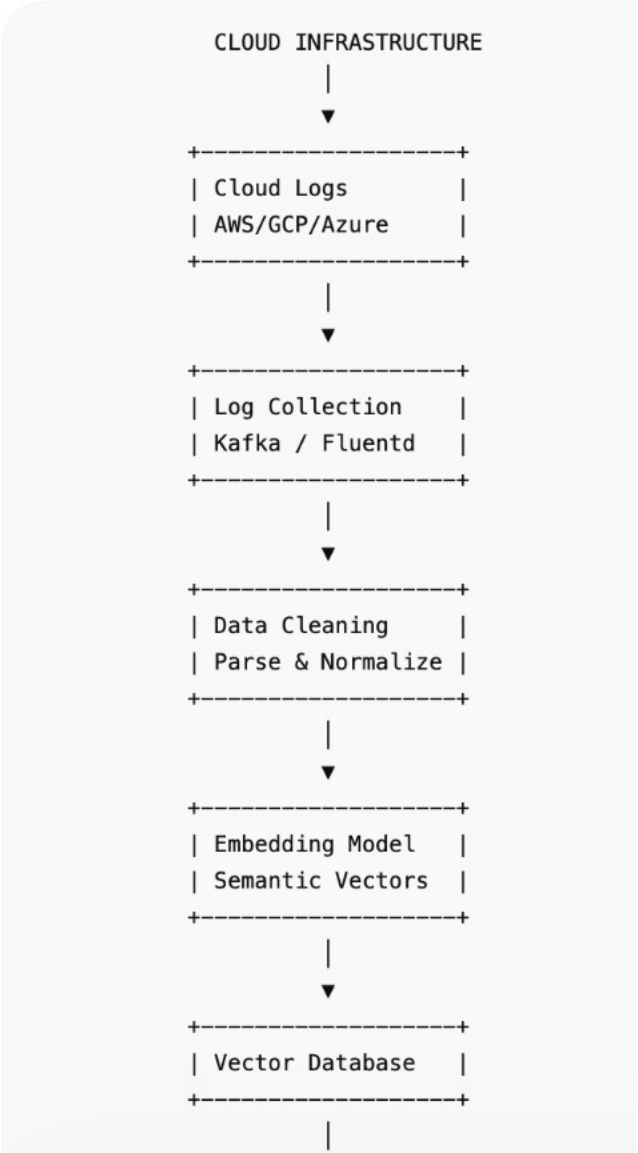
- ✗ generic
- ✗ expensive
- ✗ not company-aware
- ✗ inconsistent outputs

So companies:

- fine-tune
- use RAG
- optimize SLMs



COMPLETE ENTERPRISE FLOW



STEP-BY-STEP FLOW

STEP 1 — LOG COLLECTION

Logs come from:

- Kubernetes
- APIs
- databases
- load balancers
- cloud infra
- applications

Tools:

- Kafka
- Fluentd
- Logstash

STEP 2 — DATA CLEANING

Convert raw logs into structured format.

Example:

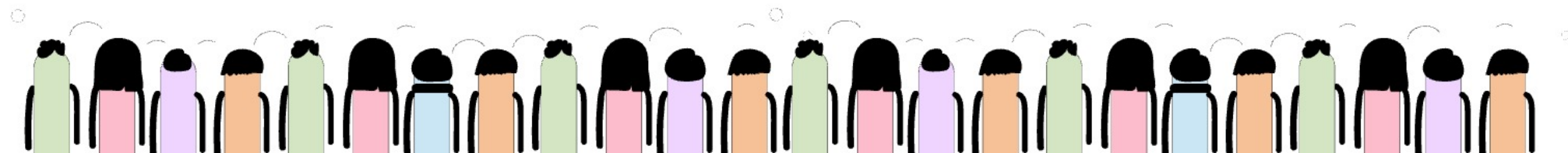
Raw:

CPU overload detected at node-17

Structured:

</> JSON

```
{  
  "node": "17",  
  "issue": "cpu_overload"  
}
```



STEP 3 — EMBEDDINGS

Convert logs into vectors.

Purpose:

- similarity search
- anomaly grouping
- semantic clustering

STEP 4 — RAG STORAGE

Store:

- incidents
- historical outages
- fixes
- architecture docs

in vector DB.

Look Up "in"

STEP 5 — FINE-TUNING

What Happens?

Train model on company-specific incidents.

TRAINING DATA EXAMPLE

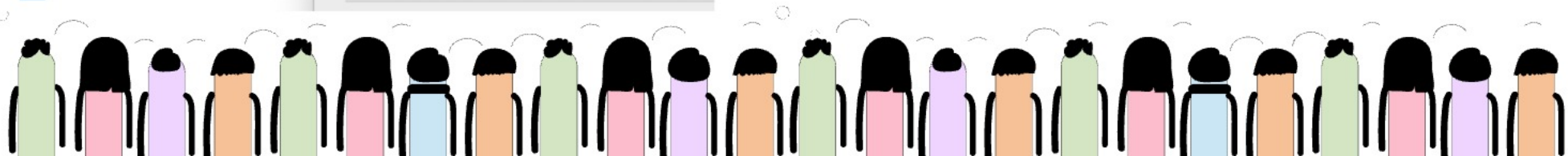
INPUT:

"API latency spike with DB timeout"

OUTPUT:

"Check database scaling and retry storm"

Thousands of examples needed.





WHICH MODELS USED?

Most companies use:

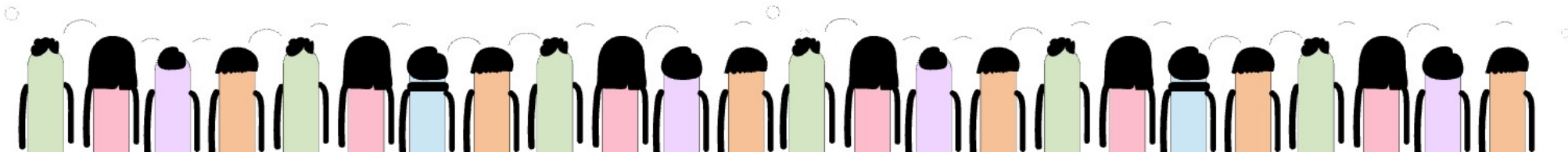
Model	Why
Phi-3	lightweight enterprise AI
Mistral 7B	open-source strong reasoning
Llama	customizable
Gemma	lightweight Google model

HOW MUCH DATA NEEDED?

Depends on task complexity.

SIMPLE ESTIMATE

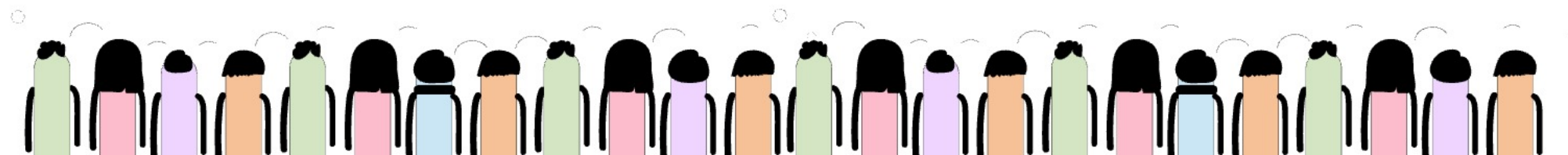
Task	Data Needed
Simple classification	1k–5k examples
Log summarization	5k–20k
Root cause analysis	20k–100k
Autonomous troubleshooting	100k+



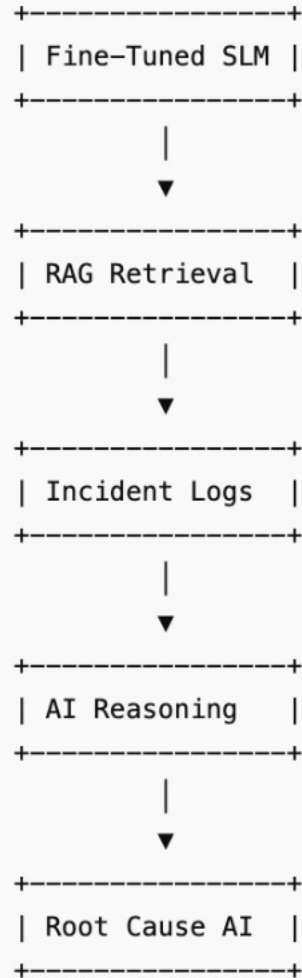


REAL ENTERPRISE MATURITY TIMELINE

Stage	Time
PoC	2–4 weeks
Good internal assistant	2–3 months
Reliable incident AI	6–12 months
Autonomous AI ops	1–3 years



BEST ENTERPRISE ARCHITECTURE

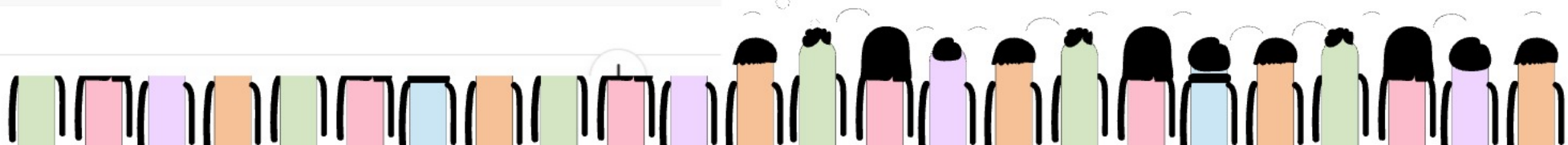


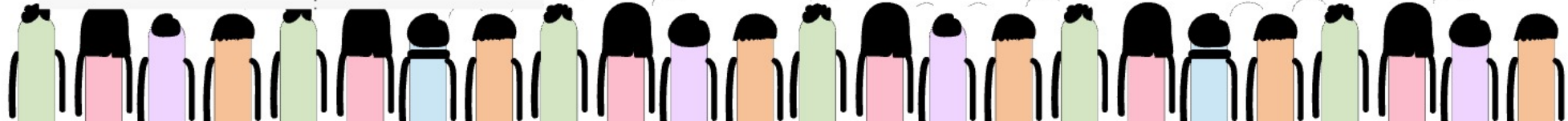
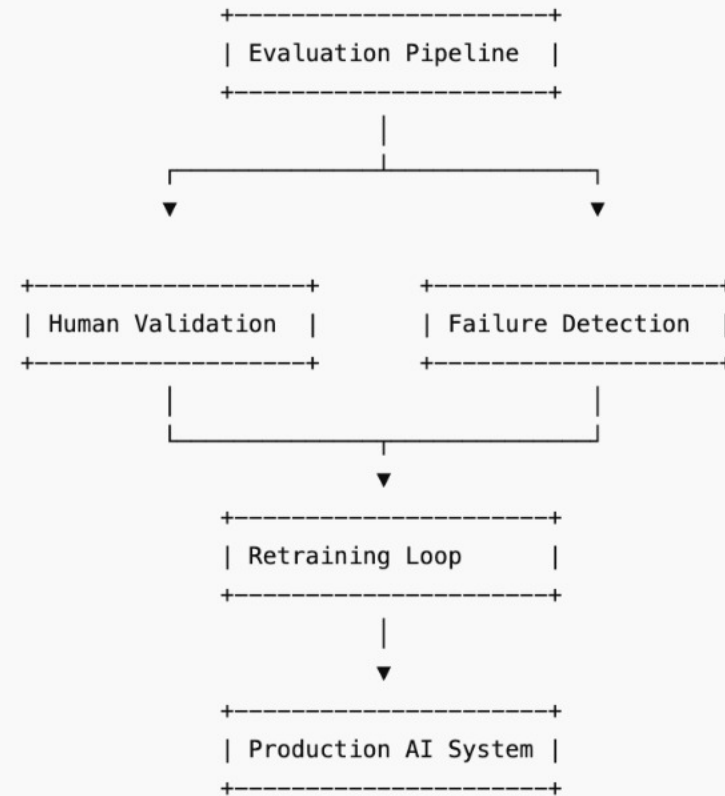
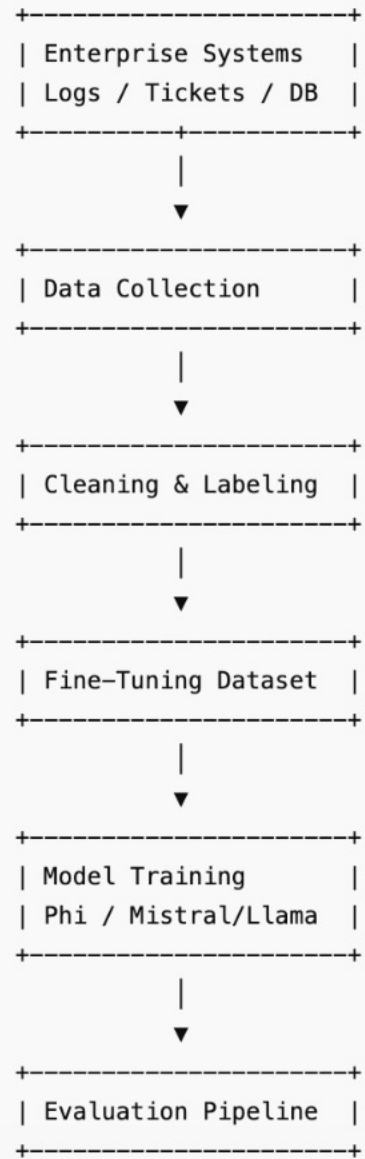
REAL ENTERPRISE FINE-TUNING JOURNEY

Year 1 → Foundation

Year 2 → Reliability & Scale

Year 3 → Autonomous Intelligence







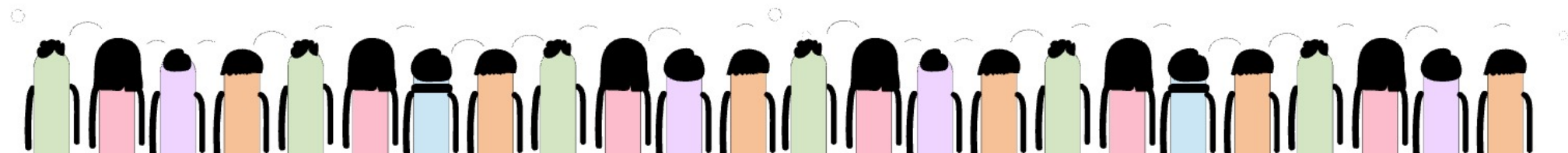
What is DSPy ?

DSPy = **Declarative Self-improving Python** framework for LLM applications

Created by Stanford University.

It helps engineers build:

- RAG systems
- AI agents
- LLM pipelines
- reasoning workflows
- prompt optimization systems





Traditional Prompting Problem

DSPy Solution

Prompt engineering messy

Structured programming

Hard to optimize prompts

Automatic optimization

Prompt changes break system

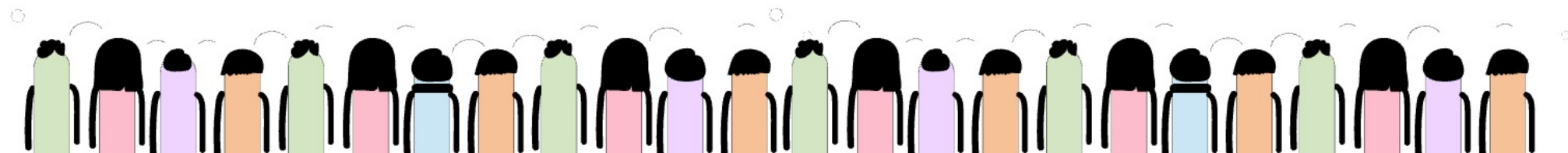
Modular architecture

Scaling agents difficult

Composable pipelines

Hard to evaluate quality

Built-in evaluation



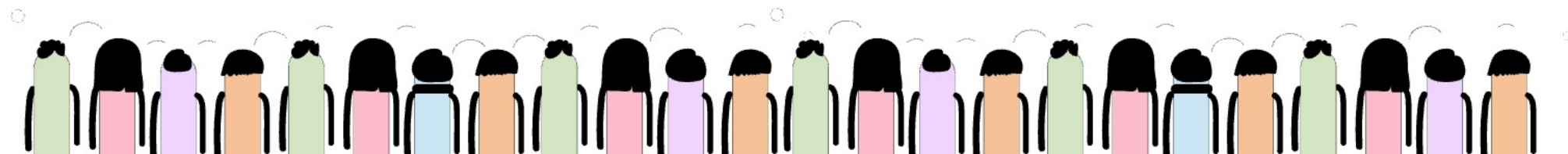


```
prompt = """  
Answer carefully...  
Use context...  
Don't hallucinate...  
"""
```

✓ DSPy way:

Python

```
class AnswerQuestion(Signature):  
    context = InputField()  
    question = InputField()  
    answer = OutputField()
```



User Query



+-----+

| DSPy Program |

+-----+



Retriever



Reasoner



Planner



Generator



Validator

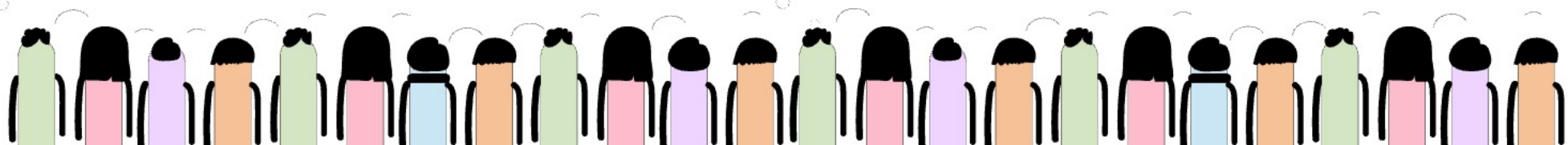
Validator



Prompt Optimization



Better Accuracy



START



Load DSPy



Configure LLM



Define AI Signatures



Build AI Pipeline



Load Training Examples



Load Training Examples



Define Evaluation Metric



BootstrapFewShot Optimizer



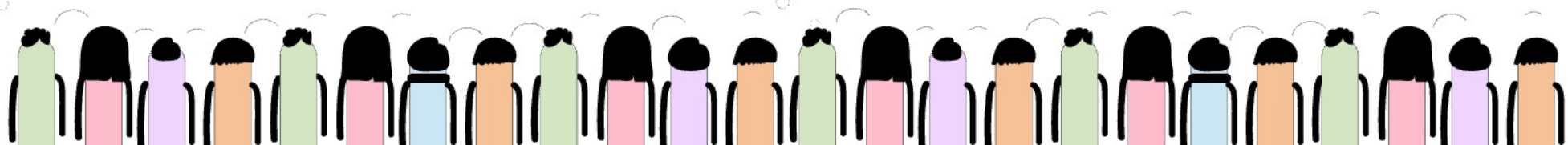
Compile Pipeline



Generate Optimized Prompts



Run User Query





▼
Step 1: Summarize Problem



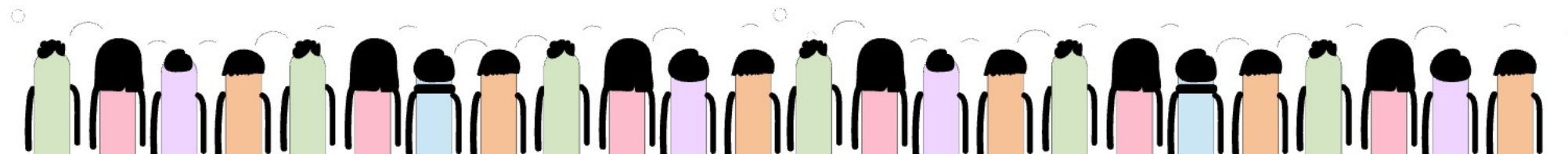
▼
Step 2: Generate Solution



▼
Return Final Solution



▼
END





1 Moving from Sequential Chains to Cyclic Graphs with LangGraph

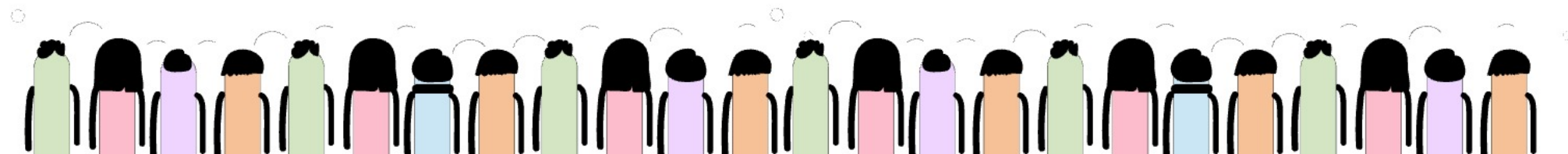
SIMPLE DEFINITION

Traditional AI pipelines:

Step 1 → Step 2 → Step 3

This is called:

Sequential Chain





SIMPLE DIFFERENCE

Sequential Chain

User Query



Retriever



LLM



Answer

Cyclic Graph

User Query



Planner Agent



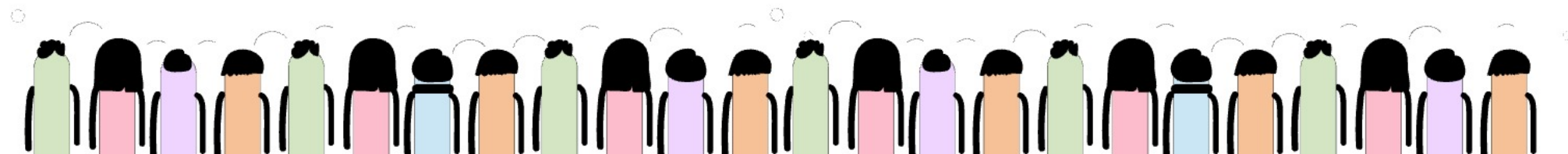
Coder Agent

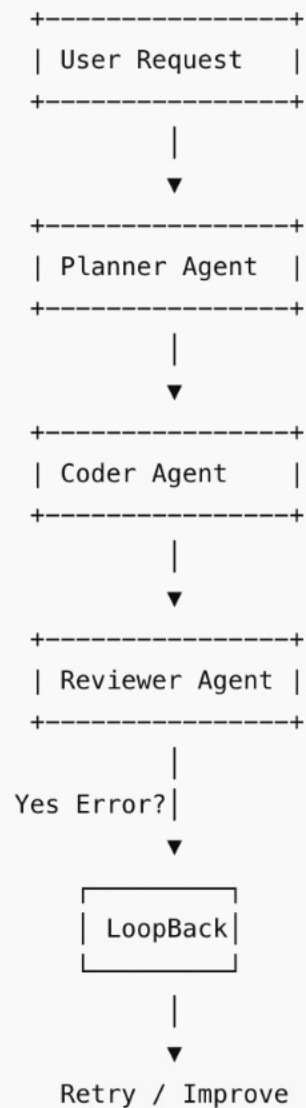


Reviewer Agent



If error?





WHEN TO USE LangGraph?

Use when:

Scenario

LangGraph Good?

Multi-agent AI



Autonomous systems



AI debugging loops



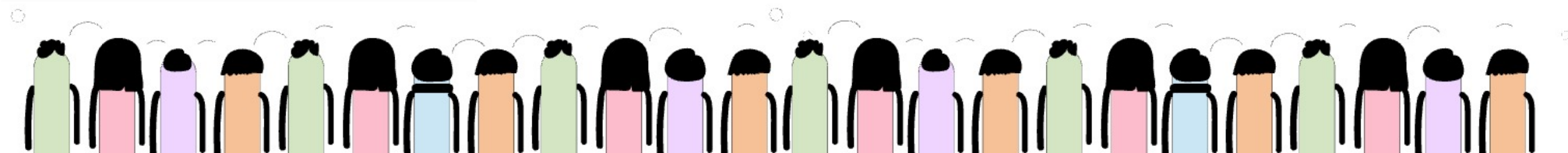
Workflow orchestration



Dynamic planning



Reflection loops





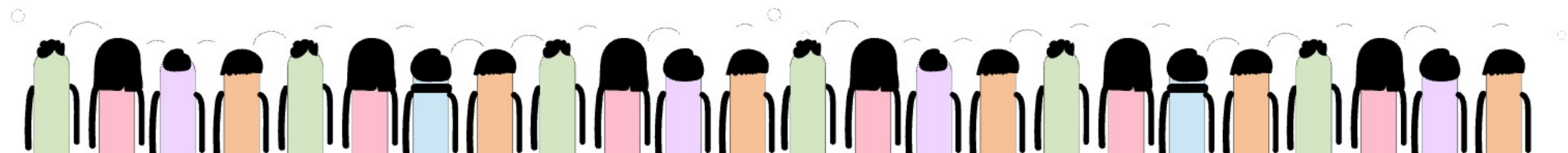
1 Collaborative Task Management with CrewAI

SIMPLE DEFINITION

CrewAI allows:

Multiple AI agents
working together
like an engineering team

Instead of ONE AI doing everything.



HUMAN TEAM VS AI TEAM

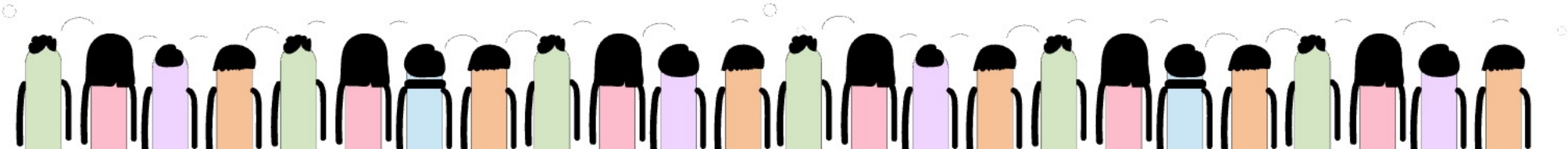


Human Software Team

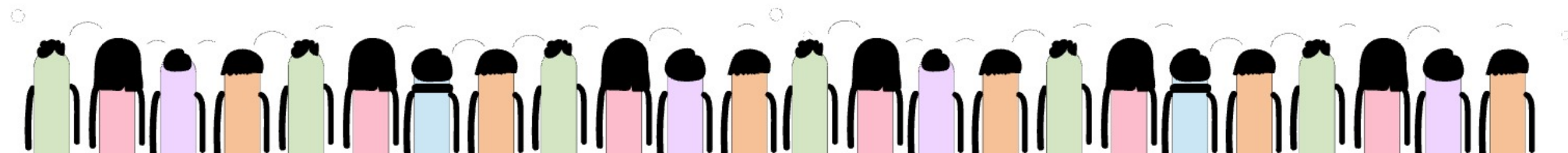
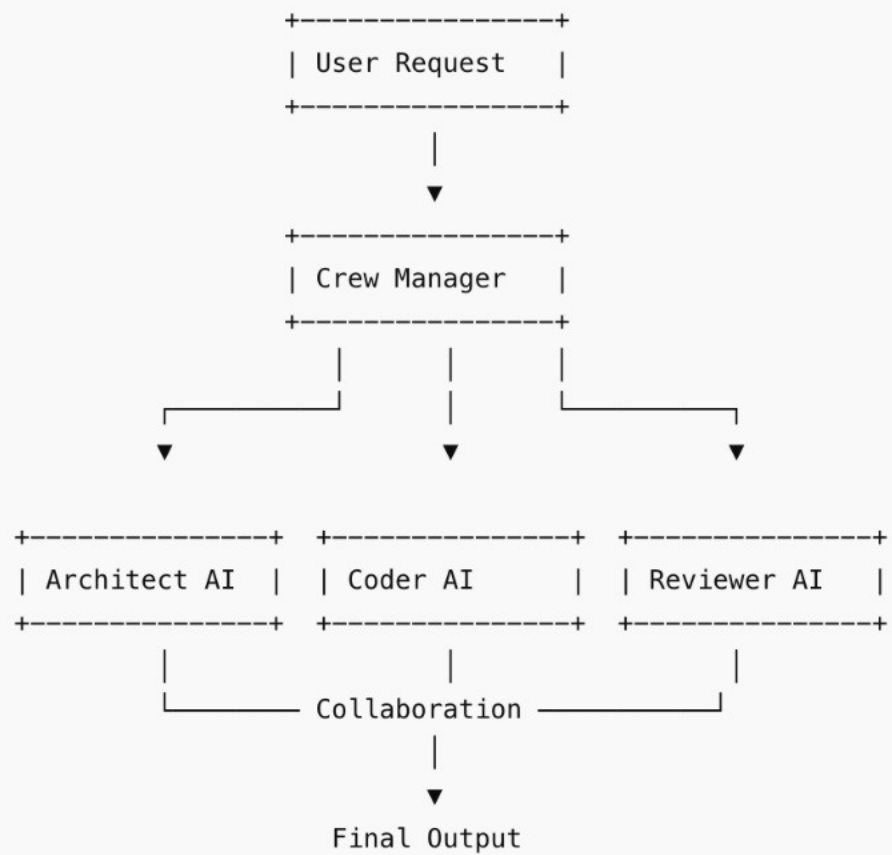
Human Role	Responsibility
Architect	design system
Developer	write code
Reviewer	validate quality
Tester	test bugs
DevOps	deployment

AI Team (CrewAI)

AI Agent	Responsibility
Architect Agent	system design
Coder Agent	implementation
Reviewer Agent	quality validation
Security Agent	vulnerability analysis
DevOps Agent	deployment planning



SIMPLE CREWAI FLOW





1 A2A Protocol (Agent-to-Agent Protocol)

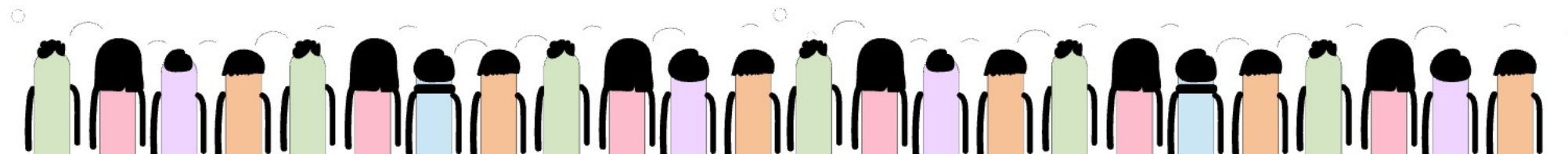
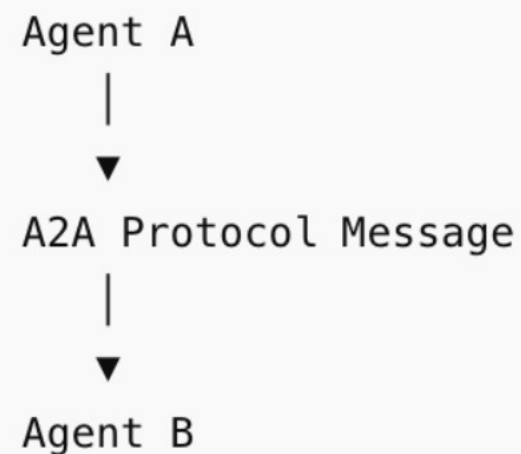
SIMPLE DEFINITION

A2A means:

Standard way for AI agents
to communicate with each other

Like APIs for software services.

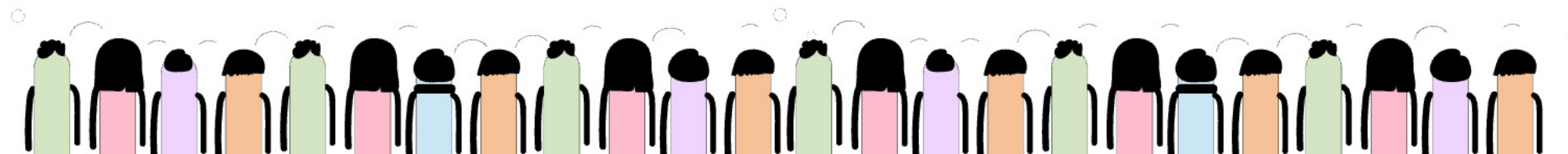
SIMPLE FLOW





REAL INDUSTRY USAGE

Industry	Usage
CloudOps	monitoring agent → repair agent
Banking	fraud agent → compliance agent
Telecom	outage detection → routing agent
Cybersecurity	threat agent → response agent
Robotics	navigation agent → control agent



REAL EXAMPLE

Cloud Infrastructure

Monitoring Agent:

"CPU overload node-17"

↓ A2A message ↓

Automation Agent:

"Scale Kubernetes pods"

SIMPLE EXAMPLE



Cybersecurity AI

Need:

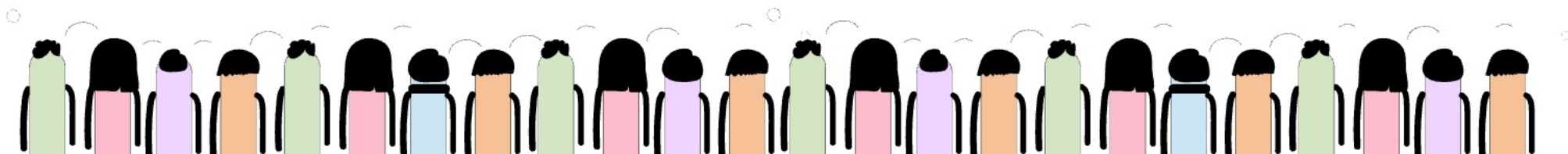
Detect attack in milliseconds

Cannot wait:

- 30 seconds
- multiple slow agent calls

AGENT LIGHTNING IDEA

Ultra-fast specialized agents
running concurrently





REAL INDUSTRY USAGE

Industry

Usage

High-frequency trading

realtime decision AI

Cybersecurity

instant threat response

Telecom

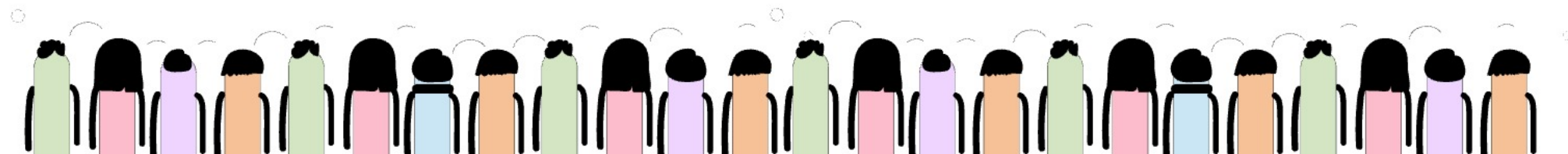
realtime network optimization

CloudOps

autoscaling agents

Robotics

realtime control systems



3 Agent Reinforcement

SIMPLE DEFINITION

Agent reinforcement means:

Agents improve themselves
using feedback loops

SIMPLE IDEA

Agent performs task.

Then system checks:

Was output good or bad?

Feedback improves future behavior.

+-----+
| Monitoring AI |
+-----+

A2A Protocol

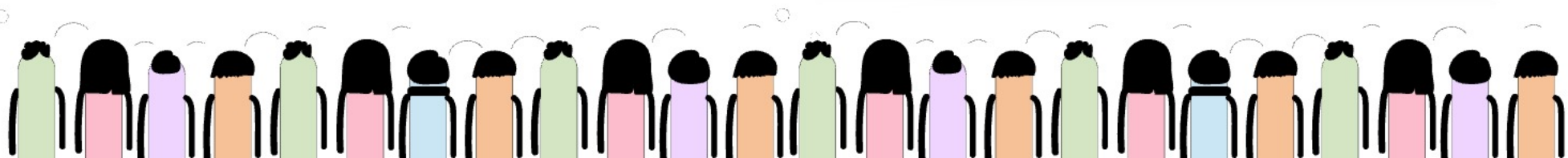
+-----+
| Diagnosis AI |
+-----+

Lightning Execution

+-----+
| Automation AI |
+-----+

+-----+
| Feedback Loop |
+-----+

Agent Reinforcement



Langfuse is:

Observability platform for AI systems

Like:

- monitoring
- logging
- tracing

for LLM applications.

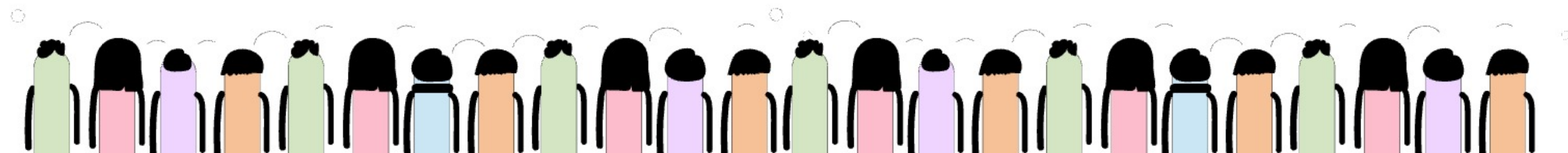
WHY CALLED "BLACK BOX"?

In production AI systems:

You don't know:

- why answer failed
- which prompt caused issue
- which agent broke
- why latency increased
- where hallucination happened

AI becomes operational black box.



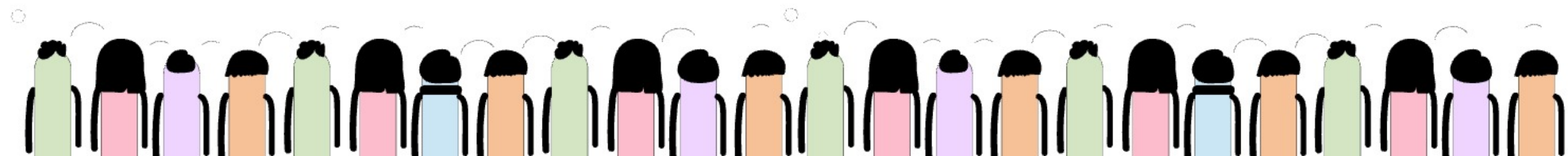
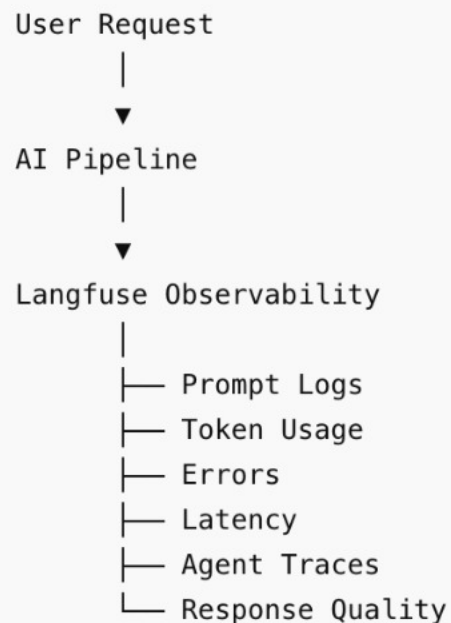


LANGFUSE SOLVES

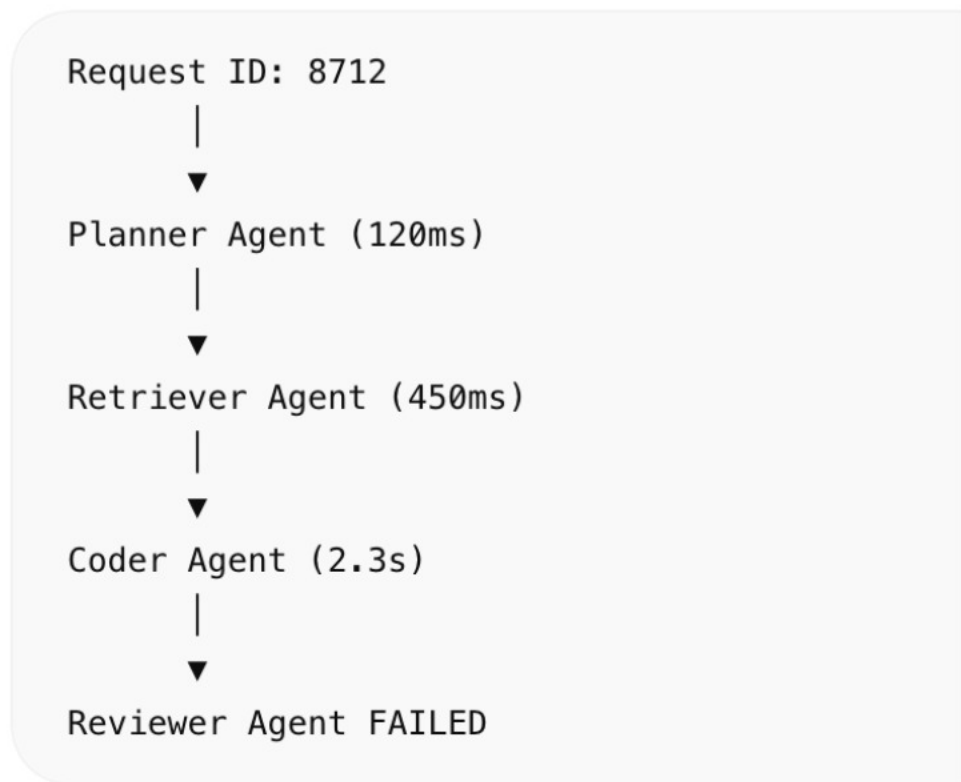
- ✓ prompt tracking
- ✓ execution tracing
- ✓ token monitoring
- ✓ cost tracking
- ✓ agent debugging
- ✓ response evaluation



SIMPLE FLOW

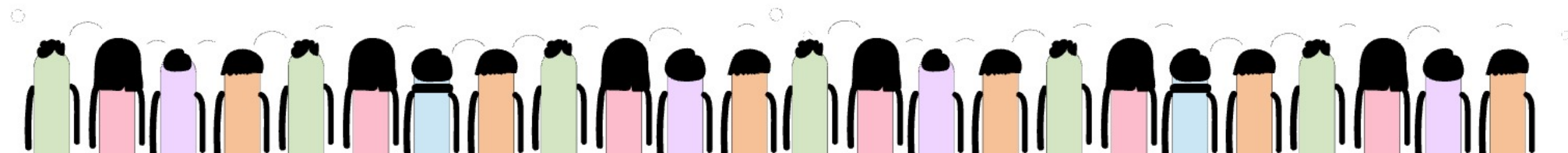
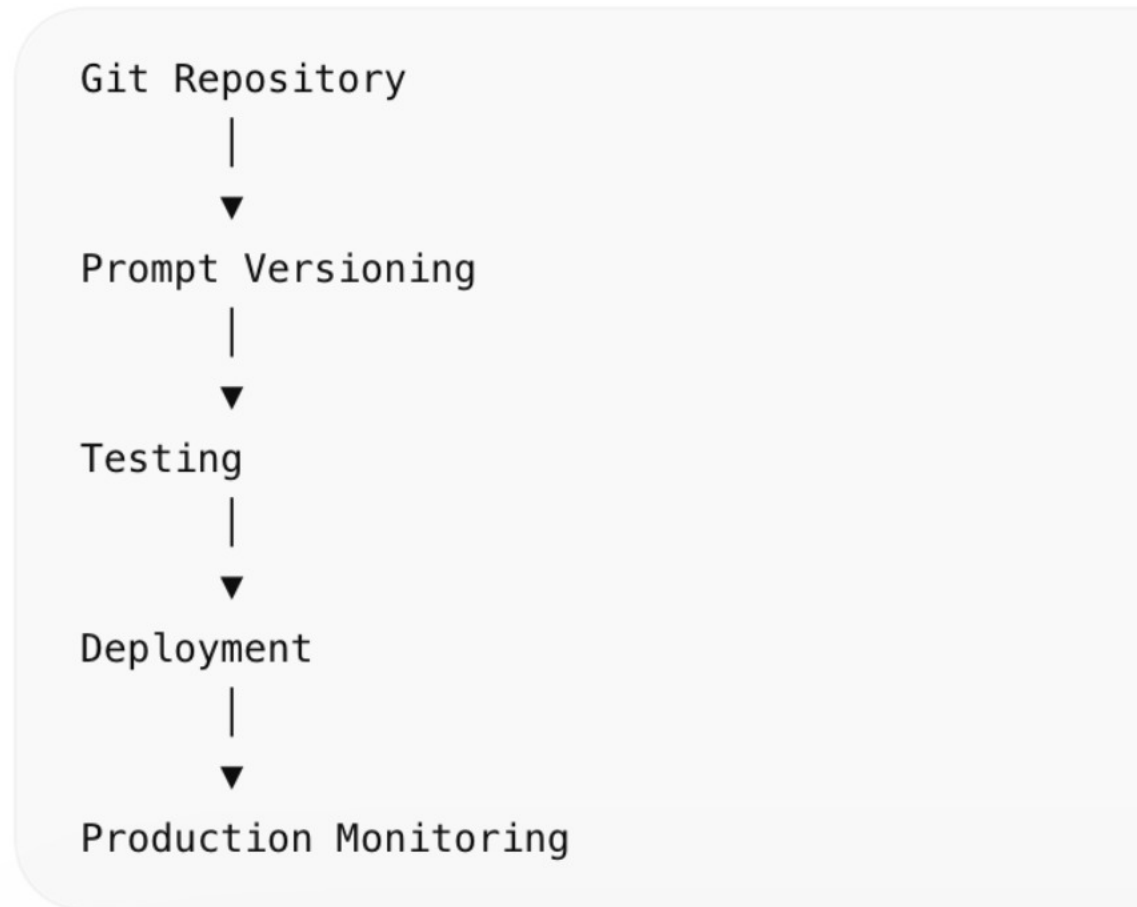


DISTRIBUTED TRACE FLOW



This is distributed tracing.

PROMPTS AS CODE FLOW





REAL INDUSTRY USAGE

Industry

Usage

Banking

compliance prompt control

Healthcare

diagnosis prompt auditing

Customer Support

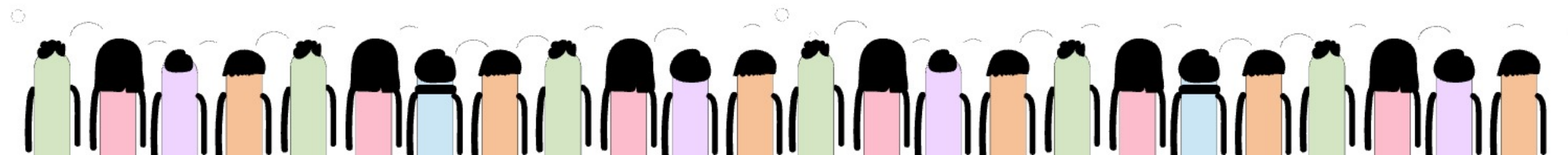
safe response rollout

Cybersecurity

controlled remediation prompts

CloudOps

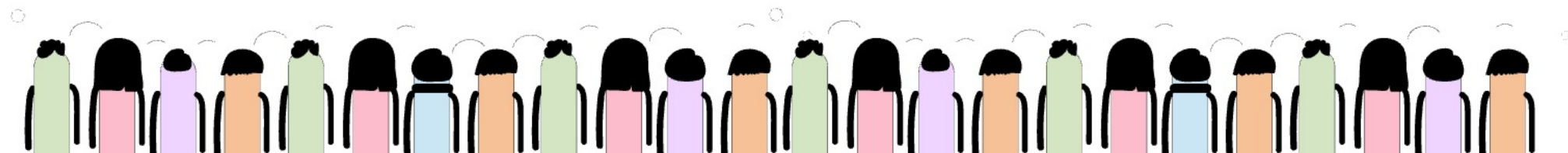
infrastructure automation prompts





"Vibe check" means:

Humans manually reading AI outputs
and saying:
"Looks good"



WHY THIS FAILS?

In enterprise AI systems:

- ✗ outputs become huge
- ✗ thousands of responses/day
- ✗ humans inconsistent
- ✗ hallucinations hidden
- ✗ quality difficult to measure

SIMPLE FLOW

AI Output



Evaluation Engine

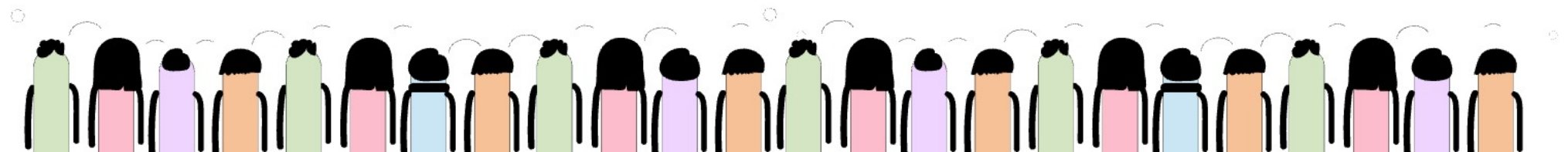


— Hallucination Score

— Faithfulness Score

— Relevancy Score

— Correctness Score





REAL INDUSTRY USAGE



Industry

Metric Evaluation Usage

Banking

compliance validation

Healthcare

diagnosis grounding

CloudOps

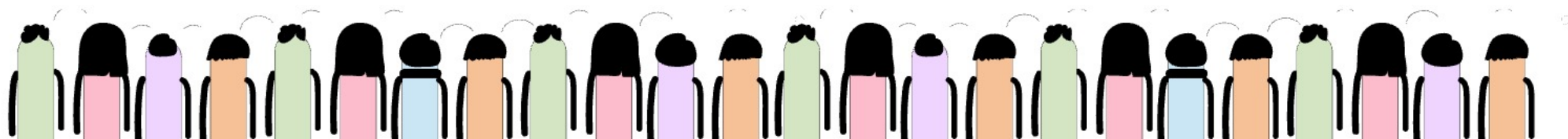
remediation accuracy

Cybersecurity

threat analysis quality

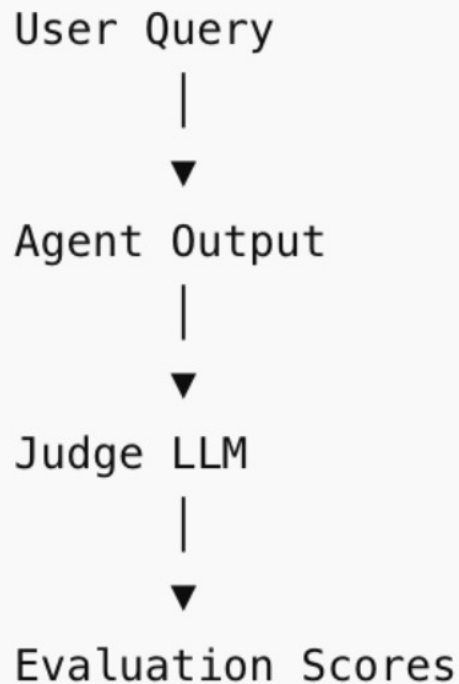
Customer Support

response correctness





SIMPLE FLOW

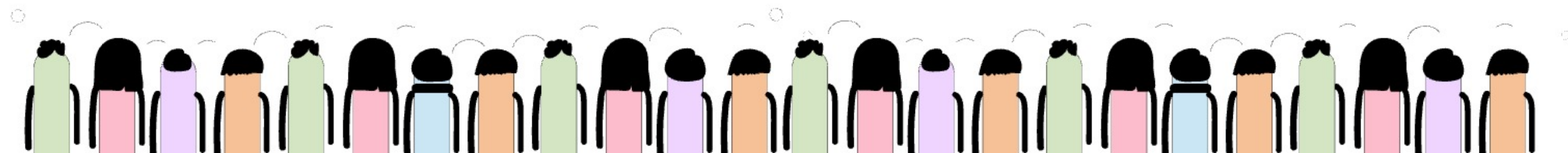


REAL MULTI-AGENT EXAMPLE



Evaluation checks:

- correct workflow?
- unnecessary loops?
- correct retrieval?
- final output quality?





Metric-Based Evaluation



Measure AI quality objectively

LLM-as-a-Judge

AI evaluates AI outputs

Hallucination Testing

Detect invented information

Faithfulness Testing

Verify grounding in source data

Relevancy Testing

Ensure answer matches question

